ChartNav — Consolidated Build

Backend phases 1–5: spine → state machine → auth → RBAC → CI/hardening
Repo: `thekidd2227/chartnav-platform`

Contents

1. Executive summary
2. 01 — Current state
3. 02 — Workflow state machine
4. 03 — API endpoints
5. 04 — Data model
6. 05 — Build log
7. 06 — Known gaps + verification matrix
8. 07 — Auth, org scoping, RBAC
9. 08 — Test strategy
10. 09 — CI & deploy hardening
11. 10 — Doc artifact pipeline
12. 11 — Production-auth seam
13. 12 — Runtime config
14. 13 — Deploy target
15. 14 — Postgres parity
16. 15 — Frontend integration
17. 16 — Frontend test strategy
18. 17 — E2E & release
19. 18 — Operational hardening
20. 19 — Staging deployment
21. 20 — Observability
22. 21 — Staging runbook
23. Diagram — System architecture
24. Diagram — Encounter status machine
25. Diagram — ER
26. Diagram — API / data flow

Executive summary

This document consolidates **all** ChartNav backend phases delivered to

date.

Phase 1 — Workflow spine. Alembic migration `a1b2c3d4e5f6` introduced `encounters` and `workflow_events` tables with FKs and indexes. Six encounter endpoints. Idempotent seed.

Phase 2 — State machine + filtering. `POST /encounters/{id}/status` enforces explicit forward + rework edges. `GET /encounters` accepts `organization_id`, `location_id`, `status`, `provider_name` filters.

Phase 3 — Dev auth + org scoping. `apps/api/app/auth.py` resolves the caller from `X-User-Email`. Every encounter route became org-scoped: cross-org reads return 404, cross-org body/query assertions 403.

Phase 4 — RBAC + full scoping + tests. `apps/api/app/authz.py` introduces `admin` / `clinician` / `reviewer` roles and per-edge transition rules. `/organizations`, `/locations`, `/users` are now authenticated and scoped. `{error_code, reason}` envelope is standardized. 25-test pytest suite covers auth, scoping, RBAC, state-machine invariants.

Phase 5 — CI + runtime hardening. GitHub Actions workflow runs install + Alembic upgrade on an isolated CI DB + idempotent seed + pytest + live smoke. A separate `docs` job regenerates the final HTML/PDF. Root `Makefile`, `apps/api/scripts/smoke.sh`, and `scripts/build_docs.py` codify the local verification + doc-build paths.

Phase 6 — Production seam + deploy target + Postgres parity. DB layer moved to SQLAlchemy Core. `apps/api/app/config.py` centralizes env. `auth.py` gained a bearer stub that returns 501 honestly. Dockerfile hardened, `docker-compose.prod.yml` + `scripts/pg_verify.sh` + `backend-

`postgres` CI job land.

Phase 7 — Frontend workflow UI. Typed API client + identity seam + full two-pane workflow console (list + filters + detail + timeline + role-aware actions). Error banners surface the backend envelope verbatim.

Phase 8 — Create UI + frontend tests + frontend CI. Admin/clinician can create encounters through a modal; Vitest + Testing Library harness (12 tests) locks the UI down; dedicated `frontend` CI job runs typecheck + tests + build on every push/PR.

Phase 9 — Playwright E2E + release pipeline. Playwright boots backend + frontend together, 8 Chromium scenarios, new `e2e` CI job. `release.yml` workflow on `v*.*.*` tags: GHCR push + release bundle + GitHub Release with tarballs and MANIFEST.

Phase 10 — Real JWT bearer auth + operational hardening. Real PyJWT validation against a JWKS URL (signature + iss + aud + exp + claim mapping). Request correlation (`X-Request-ID`), structured JSON logs, `security_audit_events` table (`b2c3d4e5f6a7`), CORS driven by config, per-process rate limiter.

Phase 11 — Staging deployment + observability (this phase). New `GET /ready` (DB-aware readiness) and `GET /metrics` (Prometheus text) surface request, auth-denied, rate-limited, audit-event, and latency counters. Staging is a pinned-image compose deploy: `infra/docker/docker-compose.staging.yml + .env.staging.example` , driven by `staging_up.sh` / `staging_verify.sh` / `staging_rollback.sh` . Release bundle now ships a `chartnav-staging-<version>.tar.gz` . New CI `deploy-config` job validates compose files + shellchecks every script. Runbook + observability docs are operator-grade (19/20/21). 51 pytest tests including 3 observability scenarios; 9-assertion staging verify runs live against a locally-booted API.

Preserved untouched: `/health` , `/` , SQLite dev workflow, state machine + filtering surface, workflow_events model, existing endpoint contracts.

ChartNav — Current State

As of: 2026-04-18 (phase: staging deployment + observability)

Repo layout (relevant)

```
chartnav-platform/
├── .github/workflows/
│   ├── ci.yml            # backend-sqlite · backend-postgres · frontend · e2e · docker-build · deploy-config · docs
│   └── release.yml       # now also bundles the staging artifact tar
├── Makefile              # + staging-up · staging-verify · staging-rollback · staging-down
├── scripts/
│   ├── build_docs.py
│   ├── verify.sh · pg_verify.sh · release_build.sh
│   └── staging_up.sh · staging_verify.sh · staging_rollback.sh
├── apps/
│   ├── api/
│   │   ├── app/
│   │   │   ├── main.py              # CORS + middleware + exception-handler audit
│   │   │   ├── config.py
│   │   │   ├── db.py
│   │   │   ├── auth.py              # real JWT bearer
│   │   │   ├── authz.py
│   │   │   ├── audit.py
│   │   │   ├── logging_config.py
│   │   │   ├── middleware.py        # request-id · access log · rate limit
│   │   │   ├── metrics.py           # NEW — in-process Prometheus counters
│   │   │   └── api/routes.py        # adds /ready, /metrics
│   │   ├── alembic/versions/        # 3 migrations through b2c3d4e5f6a7
│   │   ├── scripts_seed.py · scripts/smoke.sh
│   │   ├── tests/                   # 51 pytest (+ 3 observability)
│   │   └── Dockerfile · entrypoint.sh · .env.example
│   └── web/
│       └── (unchanged this phase)
├── infra/docker/
│   ├── docker-compose.yml           # dev
│   ├── docker-compose.prod.yml      # generic prod
│   ├── docker-compose.staging.yml   # NEW — pinned image, /ready healthcheck, volumes
│   └── .env.staging.example         # NEW — explicit staging contract
└── docs/build/ 01 … 21
```

Runtime baseline

- Backend: FastAPI + SQLAlchemy Core + PyJWT.
- Frontend: Vite 5 + React 18 + TypeScript + Vitest + Playwright.
- Auth: `header` (dev) or `bearer` (prod, real JWT with JWKS cache).
- RBAC: `admin` / `clinician` / `reviewer`.
- Error envelope: `{"detail": {"error_code": "...", "reason": "..."}}`.
- **Observability**: `/health` (liveness), `/ready` (DB-aware), `/metrics` (Prometheus text).
- **Audit trail**: `security_audit_events` table; written on 401/403 + listed error codes + 429 rate_limited.
- **CORS**: env-driven, no wildcard.
- **Rate limit**: per-process sliding window on authed paths.
- **Request correlation**: `X-Request-ID` inbound is honored, otherwise generated; always echoed.
- **Structured logs**: JSON per line.
- Alembic head: `b2c3d4e5f6a7`.

Testing layers

Layer	Tool	Count	Scope
pytest	pytest	51	backend (auth, RBAC, scoping, state machine, bearer JWT, operational, observability)
shell smoke	smoke.sh	9	live HTTP contract (SQLite + Postgres)
vitest	vitest	12	frontend integration
Playwright	@playwright	8	full-stack browser
staging smoke	staging_verify.sh	9	live staging stack (health + ready + metrics + auth + audit signal)

Deploy / release

- Release: `.github/workflows/release.yml` on `v*.*.*` tags pushes `ghcr.io/<owner>/chartnav-api:<tag> + :latest`, produces `chartnav-api-<v>.tar`, `chartnav-web-<v>.tar.gz`, `chartnav-staging-<v>.tar.gz`, and `MANIFEST.txt` in a GitHub Release.
- Staging deploy: `infra/docker/docker-compose.staging.yml + .env.staging` on the staging host; one-shot `make staging-up` / `staging-verify` / `staging-rollback TAG=...`.
- Prod: `infra/docker/docker-compose.prod.yml` remains available; it's the generic ancestor of the staging compose.

Automation

- `make verify` (backend gate), `make pg-verify` (Postgres parity), `make web-verify`, `make e2e`
- `make staging-up / staging-verify / staging-rollback TAG=... / staging-down`
- `make release-build VERSION=v0.1.0`
- `make dev` (boot backend + frontend)
- CI: `backend-sqlite + frontend + deploy-config` in parallel; `e2e` gates on backend+frontend; `backend-postgres`, `docker-build`, `docs` chain on `backend-sqlite`.

Encounter Workflow State Machine

Status values

Status	Meaning
`scheduled`	Visit is booked, patient has not arrived.
`in_progress`	Provider is seeing the patient / actively charting.
`draft_ready`	Note draft has been generated and is awaiting review.
`review_needed`	Reviewer flagged the note — needs revision or sign-off.
`completed`	Note signed off; encounter closed. **Terminal.**

Allowed transitions

Forward flow:

```
scheduled → in_progress → draft_ready → review_needed → completed
```

Explicit rework transitions (documented + enforced):

From	To	Purpose
`draft_ready`	`in_progress`	Draft rejected, return to charting.
`review_needed`	`draft_ready`	Reviewer kicks note back for rewrite.

Rejected transitions (examples that return HTTP 400)

- `scheduled → completed`
- `scheduled → review_needed`
- `scheduled → draft_ready`
- `in_progress → scheduled`
- `in_progress → completed` (must pass through draft_ready + review_needed)
- `completed → anything` (terminal)
- Any unknown status string (`invalid_status`)

Error body format:

```
{"detail":"invalid_transition: in_progress -> completed is not permitted; allowed next states from in_progress: ['draft_ready'
```

Idempotency

Posting the **same** status the encounter is already in is a no-op: the row

is returned unchanged and **no** `status_changed` event is appended.

Side effects on successful transition

Transition target	Timestamp action
`in_progress`	set `started_at = now()` if currently NULL
`completed`	set `completed_at = now()` ; also set `started_at = now()` if it was NULL
other	no timestamp mutation

Every successful transition appends a `workflow_events` row:

```
{
  "event_type": "status_changed",
  "event_data": {"old_status": "...", "new_status": "..."}
}
```

Diagram

See `docs/diagrams/encounter-status-machine.md` (Mermaid).

Source of truth

`apps/api/app/api/routes.py` :

- `ALLOWED_STATUSES` — the 5 permitted status strings.
- `ALLOWED_TRANSITIONS` — `dict[str, set[str]]` , one entry per source state.
- `update_encounter_status()` — the single enforcement point.

API Endpoints

Base URL (local dev): `http://127.0.0.1:8000` . All error bodies use

`{"detail": {"error_code": "...", "reason": "..."}}` .

System (open)

Method	Path
GET	`/health`
GET	`/`

Identity

Method	Path	Auth	Roles
GET	`/me`	🔒	any

Org metadata (🔒, org-scoped)

Method	Path	Behavior
GET	`/organizations`	Returns only caller's org row.
GET	`/locations`	`WHERE organization_id = caller.org` .
GET	`/users`	`WHERE organization_id = caller.org` .

Encounters (🔒, org-scoped + RBAC)

GET `/encounters`

Any authenticated role. Always `WHERE organization_id = caller.org` .

Optional query params (AND-ed, parameterized):

Param	Rule
`organization_id`	Must equal `caller.org` — else 403.
`location_id`	Narrows within caller org.
`status`	Must be in `ALLOWED_STATUSES` — else 400.
`provider_name`	Exact match.

GET `/encounters/{id}` — any role

404 if missing or cross-org.

GET `/encounters/{id}/events` — any role

404 if parent missing or cross-org.

POST `/encounters` — admin, clinician

Body:

```
{
  "organization_id": 1,
  "location_id": 1,
  "patient_identifier": "PT-1003",
  "patient_name": "Alex Chen",
  "provider_name": "Dr. Ortiz",
  "scheduled_at": "2026-04-18T14:30:00Z",
  "status": "scheduled"
}
```

POST `/encounters/{id}/events` — admin, clinician

Reviewer → 403. 404 if cross-org.

POST `/encounters/{id}/status`

Per-edge RBAC:

From	To	Roles allowed
scheduled	in_progress	admin, clinician
in_progress	draft_ready	admin, clinician
draft_ready	in_progress	admin, clinician
draft_ready	review_needed	admin, reviewer
review_needed	draft_ready	admin, reviewer
review_needed	completed	admin, reviewer

Same-state POST = no-op (200). Role violation → 403 `role_cannot_transition`. Invalid transition → 400 `invalid_transition`. Cross-org → 404.

Error code inventory

Code	HTTP	Origin
`missing_auth_header`	401	auth transport
`unknown_user`	401	auth transport
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to unknown
`role_forbidden`	403	generic `require_roles`
`role_cannot_create_encounter`	403	POST /encounters
`role_cannot_create_event`	403	POST /encounters/{id}/events
`role_cannot_transition`	403	POST /encounters/{id}/status
`cross_org_access_forbidden`	403	body/query asserts other org
`encounter_not_found`	404	missing or cross-org encounter
`location_not_found`	400	bad location_id
`invalid_status`	400	unknown status string
`invalid_initial_status`	400	POST /encounters bad init
`invalid_transition`	400	disallowed state-machine edge

Verification

This full surface is now locked in by two layers:

- pytest (`apps/api/tests/`) — 25 tests, per-test ephemeral SQLite.
- Live smoke (`apps/api/scripts/smoke.sh`) — 9 assertions against a running API.

Both run in CI on every push/PR. See `09-ci-and-deploy-hardening.md`.

Data Model

SQLite (local dev). Schema produced by Alembic migrations

`43ccbf363a8f → a1b2c3d4e5f6` . No schema changes in this RBAC/scoping phase.

Tables (unchanged this phase)

`organizations`

column	type	constraints
id	INTEGER	PK
name	VARCHAR(255)	NOT NULL
slug	VARCHAR(255)	NOT NULL, UNIQUE
created_at	DATETIME	NOT NULL default now()

`locations`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
name	VARCHAR	NOT NULL
created_at	DATETIME	NOT NULL default now()

`users`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
email	VARCHAR	NOT NULL, UNIQUE
full_name	VARCHAR	NULL
role	VARCHAR	NOT NULL default `"admin"`
created_at	DATETIME	NOT NULL default now()

Application-level role vocabulary (enforced by `app/authz.py`):

`admin` , `clinician` , `reviewer` . The column remains a free VARCHAR in

SQL; constraint is enforced at the app layer. If you need a DB-level

CHECK constraint, add it in a future migration.

`encounters` and `workflow_events`

Unchanged.

`security_audit_events` (added in phase 10)

column	type	constraints
id	INTEGER	PK

column	type	constraints
event_type	VARCHAR(100)	NOT NULL, indexed
request_id	VARCHAR(64)	NULL
actor_email	VARCHAR(255)	NULL, indexed
actor_user_id	INTEGER	NULL
organization_id	INTEGER	NULL
path	VARCHAR(512)	NULL
method	VARCHAR(16)	NULL
error_code	VARCHAR(100)	NULL
detail	TEXT	NULL
remote_addr	VARCHAR(64)	NULL
created_at	DATETIME	NOT NULL default now(), indexed

No FKs — audit rows must survive user/org deletion. Populated only on denied or suspicious access (see `18-operational-hardening.md`).

Migration: `b2c3d4e5f6a7` .

Seeded tenants (two, with full role coverage)

Org 1 — `demo-eye-clinic` (id=1)

Location: `Main Clinic` (id=1).

email	role
admin@chartnav.local	admin
clin@chartnav.local	clinician
rev@chartnav.local	reviewer

Encounters:

id	patient	provider	status	events
1	PT-1001	Dr. Carter	`in_progress`	3
2	PT-1002	Dr. Patel	`review_needed`	5

Org 2 — `northside-retina` (id=2)

Location: `Northside HQ` (id=2).

email	role
admin@northside.local	admin
clin@northside.local	clinician

Encounters:

id	patient	provider	status	events
3	PT-2001	Dr. Ahmed	`scheduled`	1

Relationships

Unchanged — see `docs/diagrams/er-diagram.md` .

Event taxonomy

event_type	Writer	Shape
`encounter_created`	`POST /encounters` / seed	`{"status": "...", "created_by": "<email>"}`
`status_changed`	`POST /encounters/{id}/status`	`{"old_status": "...", "new_status": "...", "changed_by": "<email>"}`
user-supplied	`POST /encounters/{id}/events`	Any JSON

05 — Build log

Build Log

Reverse-chronological.

2026-04-18 — Phase 11: staging deployment + observability

Step 1 — Baseline

- Head: `cbc5184` (real JWT + operational hardening).
- 48/48 pytest, 9/9 smoke green. Rate limiter / audit / structured logs all in place.

Step 2 — Observability surfaces

- New `apps/api/app/metrics.py` : in-process Prometheus-text counters behind a threading lock. Counters are narrow on purpose:
- `chartnav_requests_total{method,path,status}`
- `chartnav_auth_denied_total{error_code}`
- `chartnav_rate_limited_total`
- `chartnav_audit_events_total{event_type}`
- `chartnav_http_request_duration_ms_{sum,count}`
- `middleware.py::AccessLogMiddleware` now observes every response's `(method, path, status bucket, duration_ms)` .
- `middleware.py::RateLimitMiddleware` now bumps `rate_limited_total` before it writes the audit row.
- `audit.py::record(...)` now bumps `audit_events_total{event_type}` before inserting. Fails-closed still applies — metric or insert failure cannot mask the original 4xx.
- `main.py::_http_exception_handler` now bumps `auth_denied_total{error_code}` alongside the audit + WARNING log it already emits.
- New `GET /ready` — runs `SELECT 1` against the DB, returns `503 not_ready` if it fails. Designed to back container healthchecks.
- New `GET /metrics` — unauthed Prometheus text exposition. Documented as "internal network only" in `20-observability.md` .

Step 3 — Staging deployment artifacts

- New `infra/docker/docker-compose.staging.yml` : pinned image (`CHARTNAV_IMAGE_TAG`), 127.0.0.1-bound ports (reverse proxy expected), DB healthcheck, API healthcheck on `/ready` , named volume for Postgres, `restart: unless-stopped` , every critical env var blocked with `${VAR:?msg}` .
- New `infra/docker/.env.staging.example` : the full staging contract (image owner/tag, DB creds, bearer auth, CORS, rate-limit, seed, API port). Annotated with guardrails ("NEVER put `*` in CORS", etc.).
- New runbook scripts:
- `scripts/staging_up.sh` — validates compose config, optional `--pull` , then `up -d` .
- `scripts/staging_verify.sh` — health + ready + metrics + unauth 401 + request-id round-trip + (header-mode only) full workflow mutation + audit signal assertion.
- `scripts/staging_rollback.sh <prev_tag>` — atomic rewrite of `CHARTNAV_IMAGE_TAG` via python shim, `docker compose pull api` , `up -d api` , poll `/ready` ≤40s.
- All three scripts pass `shellcheck` clean.

Step 4 — Release bundle expansion

- `scripts/release_build.sh` now also tars the staging artifact set (`docker-compose.staging.yml` + `.env.staging.example` + all three runbook scripts + docs 19/20/21) into `chartnav-staging-<version>.tar.gz` .
- `MANIFEST.txt` sha256s all three artifacts.
- `release.yml` attaches the staging tarball to tag-based GitHub Releases.

Step 5 — CI: `deploy-config` job

- New `deploy-config` lane in `.github/workflows/ci.yml` :
- `docker compose config` on all three compose files (dev / staging / prod).
- `shellcheck` on every repo script.
- Runs on every push/PR. A broken compose file or sloppy script lands a red check before an operator ever hits `staging-up` .

Step 6 — Makefile

- New targets: `staging-up`, `staging-verify`, `staging-rollback TAG=...`, `staging-down`.

Step 7 — Tests

- New `apps/api/tests/test_observability.py` (3 tests):
- `/ready` returns 200 with `database=ok`.
- `/metrics` is Prometheus text, exposes the expected series, and reflects an `auth_denied` after an unauth `/me`.
- `/metrics` `chartnav_rate_limited_total` ticks when a request crosses the configured limit.
- Full suite now **51/51 passing**.

Step 8 — Verification

- `make verify` → 51 pytest + 9 smoke, clean teardown.
- Ran `staging_verify.sh` against a live `uvicorn` on `:8000` — all 9 assertions green including the audit/metrics check.
- `docker compose --env-file .env.staging.test config` against the staging compose file parses (normalized output prints cleanly).
- Both `ci.yml` and `release.yml` YAML parse.
- Docs regenerated.

Step 9 — Hygiene

- `.gitignore` already excludes `*.db`, caches, `dist/release/`. Staging runbook mentions never committing `.env.staging` (real values); the `.example` file stays committed.

Prior phases

- **Phase 10 — Real JWT bearer + operational hardening** (`cbc5184`)
- **Phase 9 — Playwright E2E + release pipeline** (`74fe8dd`)
- **Phase 8 — Create UI + vitest + frontend CI** (`f83d748`)
- **Phase 7 — Frontend workflow UI** (`c4f6e4f`)
- **Phase 6 — Prod auth seam + Docker + Postgres parity** (`700bb0b`)
- **Phase 5 — CI + runtime hardening + doc pipeline** (`cfa8ca9`)
- **Phase 4 — RBAC + full scoping + pytest** (`c6f29e6`)
- **Phase 3 — Dev auth + org scoping** (`efb5b56`)
- **Phase 2 — Strict state machine + filtering** (`505f025`)
- **Phase 1 — Workflow spine** (`93fceb4`)

Known Gaps & Verification Matrix

Verification evidence — phase 11

Local gates

Gate	Result
`make verify` (backend)	✅ **51/51 pytest + 9/9 smoke**
`bash scripts/staging_verify.sh` (live API)	✅ 9/9 assertions
`docker compose … config` (dev / staging / prod)	✅ all three parse
`shellcheck` on all scripts	✅ clean (0 findings)
YAML parse on `ci.yml` + `release.yml`	✅
`scripts/build_docs.py` → HTML + PDF	✅ regenerated

pytest summary (51)

Suite	Count	Notes
`test_auth.py`	5	header mode
`test_auth_modes.py`	11	header + real JWT bearer
`test_rbac.py`	12	role-gated writes + per-edge transitions
`test_scoping.py`	8	org scoping + cross-org denial
`test_operational.py`	12	request id, audit trail, rate limit, CORS
`test_observability.py`	**3**	**`/ready`, `/metrics` text, `/metrics` rate-limited counter**

Staging verify matrix (9)

Assertion	Result
`GET /health` 200	✅
`GET /ready` 200 and `database=ok`	✅
`GET /metrics` 200 and exposes `chartnav_requests_total`	✅
`GET /me` without auth → 401	✅
`X-Request-ID` round-trips	✅
Header-mode: `GET /me` with admin → 200	✅
Header-mode: `GET /encounters` with admin → 200	✅
Header-mode: `POST /encounters/1/events` → 201 (or skip if no seed)	✅
`/metrics` reflects a `missing_auth_header` denial	✅

Postgres parity

Not rerun this phase — no backend SQL changes since the last `make pg-verify` pass. Additive audit-table migration uses SA-portable constructs.

Honest limitations

- CI has no `act`-style local workflow runner available in the dev shell; `deploy-config` was verified by running its component commands manually. The YAML parses cleanly.

- `staging_verify.sh` bearer path: the script can't mint real tokens, so in `CHARTNAV_AUTH_MODE=bearer` it only covers unauth surfaces + observability. The manual bearer path is documented in `21-staging-runbook.md` .
- Metrics & rate limiter are **per-process**; multi-worker deployments will see counters and limits split per worker. Edge aggregation is required for coordinated behavior.

Real gaps (prioritized for next phase)

1. **No OpenTelemetry / distributed tracing** — correlation is by `request_id` only.

2. **No log shipping / retention defined** — stdout JSON is ready; operators wire the collector.

3. **No audit-table archival / retention policy.**

4. **Metrics are per-process** — multi-worker or multi-node needs a proper Prometheus multiprocess mode or a push gateway.

5. **Rollback cannot reverse destructive migrations** (forward-only policy; documented in `21-staging-runbook.md`).

6. **No automated staging deploy from CI** — tag pushes publish the image but the `make staging-up` handoff is manual. That's intentional until there's a protected target + secrets wired.

7. **No production equivalent of `staging_verify.sh`** yet. Extending it is trivial once the prod target exists.

8. **No JWKS-rotation test** — PyJWKClient caches aggressively; rotation is handled by the IdP.

9. **No refresh-token / revocation flow.**

10. **No signing / SBOM / provenance** on release artifacts.

11. **`/organizations`** , **`/locations`** , **`/users`** still read-only.

12. **`users.role`** free VARCHAR at DB layer.

13. **No pagination** on `GET /encounters` .

14. **Free-form `event_data`** — no per-event_type schema.

15. **No visual-regression / a11y audits** on the frontend.

16. **pytest matrix on Postgres** not wired — fixture env-driven, ready to flip.

Auth, Org Scoping, RBAC

1. Authentication (authn)

Source: `apps/api/app/auth.py`.

- Transport today: request header `X-User-Email: <email>`.
- `require_caller` is a FastAPI dependency that resolves the user from

the `users` table and returns a `Caller(user_id, email, full_name, role, organization_id)`.

- Environment flag `CHARTNAV_AUTH_MODE` (default `"header"`) is the

production upgrade seam. When JWT/SSO lands, only the body of

`require_caller` (or a new branch in it) changes. Every route and

every RBAC helper continues to work unchanged because they depend on

`Caller`, not on any header.

Dev-only, explicitly

`X-User-Email` is trivially spoofable. This is acceptable for local

development only. **Do not deploy this to a shared environment

without first swapping the transport.** The seam is there; use it.

Standardized error envelope

Every auth/authz error returns:

```
{"detail": {"error_code": "<stable_code>", "reason": "<human message>"}}
```

Authn error codes

Code	HTTP	When
`missing_auth_header`	401	`X-User-Email` absent or empty.
`unknown_user`	401	Email not found in `users`.
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to something unwired.

2. Authorization (authz / RBAC)

Source: `apps/api/app/authz.py`.

Roles

Role	Intent
`admin`	Full read/write inside own org.
`clinician`	Charting side of the workflow.
`reviewer`	Review side of the workflow; read-only on create/events.

All seeded users today carry exactly one of these roles.

Permission surface

Surface	admin	clinician	reviewer
Read org / locations / users	✓	✓	✓
List / read encounters	✓	✓	✓
Read encounter events	✓	✓	✓
Create encounter	✓	✓	×
Add workflow event	✓	✓	×
Transition scheduled→in_progress	✓	✓	×
Transition in_progress→draft_ready	✓	✓	×
Transition draft_ready→in_progress (rework)	✓	✓	×
Transition draft_ready→review_needed	✓	×	✓
Transition review_needed→draft_ready (kick back)	✓	×	✓
Transition review_needed→completed	✓	×	✓

Authz dependencies

- `require_roles(*roles)` — generic gate, returns `Caller`.
- `require_create_encounter` — applied to `POST /encounters`.
- `require_create_event` — applied to `POST /encounters/{id}/events`.
- `assert_can_transition(caller, from, to)` — called by the status

handler **after** the state machine accepts the edge.

Authz error codes

Code	HTTP	When
`role_forbidden`	403	Generic `require_roles` denies.
`role_cannot_create_encounter`	403	Non-admin/clinician POSTs `/encounters`.
`role_cannot_create_event`	403	Non-admin/clinician POSTs event.
`role_cannot_transition`	403	Role may not drive that specific state edge.

3. Org scoping

Caller's `organization_id` is **authoritative**. Bodies and query

strings cannot override it.

Route	Scoping
`GET /organizations`	`WHERE id = caller.org`
`GET /locations`	`WHERE organization_id = caller.org`
`GET /users`	`WHERE organization_id = caller.org`
`GET /encounters`	`WHERE organization_id = caller.org` + filters
`GET /encounters/{id}`	404 if row is cross-org
`GET /encounters/{id}/events`	404 if parent cross-org
`POST /encounters`	Body `organization_id` must equal caller's; location must belong
`POST /encounters/{id}/events`	404 if cross-org
`POST /encounters/{id}/status`	404 if cross-org

404 vs 403

- **404** when the target might-or-might-not exist in another org —

returning 403 there would leak existence.

- **403** when the client explicitly asserts a different org (body or

`?organization_id=`) — the intent is unambiguous, fail loudly.

4. Diagram

5. Status as of phase 10

- **JWT bearer mode is now implemented** (see `11-production-auth-seam.md`). Real signature + issuer + audience + expiry validation against a JWKS URL.
- **Dedicated audit trail** in `security_audit_events` for denied/suspicious access (see `18-operational-hardening.md`).
- **Rate limiting** on authed paths (see `18-operational-hardening.md`).
- Still TODO: scoped writes to organizations/locations/users, per-tenant key management.

6. What this phase explicitly does NOT do

- No password auth (IdP owns identity proof).
- No refresh-token flow or revocation list.
- No per-tenant JWKS — single issuer + single JWKS URL.

Test & Smoke Strategy

Stack

- `pytest` + FastAPI `TestClient` for unit/integration.
- Per-test temporary SQLite (fresh migrate + seed per test); env-based wiring so the same fixture will later work against Postgres.
- Live shell smoke (`apps/api/scripts/smoke.sh`) used by:
- `make verify` (SQLite)
- `scripts/pg_verify.sh` (Postgres)
- the built Docker image smoke job
- CI runs all of the above on every push / PR.

Layout

```
apps/api/tests/
├── conftest.py          # env-based per-test DB, header presets
├── test_auth.py         # authn surface (5)
├── test_auth_modes.py   # header + bearer seam (3)
├── test_rbac.py         # role-gated writes + state transitions (12)
└── test_scoping.py      # org scoping (8)

apps/api/scripts/
└── smoke.sh             # curl-level live smoke (9 checks)

scripts/
├── verify.sh            # boot + smoke + teardown (SQLite)
└── pg_verify.sh         # same flow against a throwaway Postgres
```

Coverage

Surface	pytest	smoke	Postgres smoke	Docker-image smoke
`/health` , `/`	✓	✓	✓	✓
Header-mode auth	✓	✓	✓	✓
Bearer-mode seam (501 / config)	✓			
Org scoping	✓	✓	✓	✓
RBAC	✓			
Filter on `/encounters`	✓			
State machine invariants	✓		✓	
Event provenance (`changed_by`)	✓		✓	
Migrations			✓	✓
Seed idempotency (x2 runs)			✓	

How to run

```
# Fast pytest (SQLite)
cd apps/api && pytest tests/ -v

# Full local SQLite gate
```

```
make verify                # resets DB, pytest, boot, smoke, teardown

# Postgres parity (docker required)
make pg-verify

# Docker image + smoke
make docker-build
```

Results at time of writing

- `pytest` : **28 passed in ~19s**
- `make verify` (SQLite): 28 pytest + 9 smoke — all green
- `scripts/pg_verify.sh` : migrations + seed (x2) + smoke + status transition + event write — **PASS**

Backend coverage (phase 10)

- 48 pytest tests across `test_auth` , `test_auth_modes` , `test_rbac` , `test_scoping` , `test_operational` .
- Real JWT validation via a locally generated RSA keypair; no external IdP.
- Audit + request-id + rate-limit + CORS all asserted at the integration level.

Frontend verification (phase 8)

- `make web-verify` = `npm run typecheck` + `npm test` + `npm run build` .
- Vitest + Testing Library + jsdom. 12 integration tests mocking

`./api` . Full matrix in `16-frontend-test-strategy.md` and

`06-known-gaps.md` .

- Vite production build still emits `dist/` in ~1s.
- CI now runs a dedicated `frontend` job on every push/PR: node 20,

`npm ci` , typecheck, test, build.

Gaps not yet covered

- pytest matrix against Postgres (the fixture is already env-driven).
- Concurrent writers / race conditions.
- `event_data` schema per `event_type` .
- Pagination behavior (not yet implemented).
- Bearer-mode JWT validation (placeholder only; see `11-production-auth-seam.md`).

CI & Deploy Hardening

GitHub Actions

File: `.github/workflows/ci.yml`. Triggers on **push to `main`** and **every pull request**.

`backend-sqlite`

Install deps → `alembic upgrade head` against an isolated `$RUNNER_TEMP/chartnav_ci.db` via `DATABASE_URL` → seed **twice** (idempotency proof) → `pytest tests/ -v` (28 tests) → `bash scripts/verify.sh` (boots uvicorn, runs `scripts/smoke.sh`).

`backend-postgres` (needs: `backend-sqlite`)

Service container `postgres:16-alpine` on localhost:5432. Install deps (incl. `[postgres]` extra) → `alembic upgrade head` on Postgres → seed twice → boot uvicorn → `scripts/smoke.sh` → live status transition asserts the returned row. Fails the workflow if Postgres parity breaks.

`frontend`

Node 20 + npm cache keyed on `apps/web/package-lock.json` → `npm ci` → `npm run typecheck` → `npm test` (vitest, 12 integration tests) → `npm run build`. Runs in parallel with `backend-sqlite` — the frontend is a peer quality gate, not blocked by backend CI.

`deploy-config`

Independent — runs in parallel with the backend/frontend gates.

- `docker compose config` on all three compose files (dev/staging/prod).
- `shellcheck` on every repo script (`scripts/*.sh`, `apps/api/scripts/smoke.sh`).

A broken compose file or sloppy script lands a red check before anyone runs `staging-up`.

`e2e` (needs: `backend-sqlite`, `frontend`)

Python 3.11 + Node 20 installed. `pip install -e "apps/api[dev,postgres]"` + `npm ci` in `apps/web`. `npx playwright install --with-deps chromium`. Then `npx playwright test --reporter=list` — Playwright boots backend on 8001 (SQLite) and frontend on 5174, runs the 8 browser scenarios, tears both down. On failure, `playwright-report/` and `test-results/` are uploaded as a workflow artifact for triage.

`release.yml` (separate workflow)

Triggers on `v*.*.*` tag push and manual dispatch. Builds + pushes `ghcr.io/<owner>/chartnav-api:<version>` (+ `:latest`), runs `scripts/release_build.sh` to produce `dist/release/<version>/` (docker-saved image tar, web bundle tar.gz, MANIFEST with sha256s), uploads the directory as an artifact, and on tag pushes attaches the files to a GitHub Release with auto-generated notes. Full reference in `17-e2e-and-release.md`.

`docker-build` (needs: `backend-sqlite`)

Buildx → build `chartnav-api:ci` from `apps/api/` → run the container with `DATABASE_URL=sqlite:///./chartnav.db` and `CHARTNAV_RUN_SEED=1` → poll `/health` → run `scripts/smoke.sh` against the live container. Proves the production image boots end-to-end.

`docs` (needs: `backend-sqlite`)

apt-install Chromium → `python scripts/build_docs.py` with `CHARTNAV_PDF_BROWSER=chromium-browser` → upload HTML + PDF as `chartnav-docs-final` artifact (`if-no-files-found: error`).

What CI catches

Change	Failure surface
Broken migration	alembic step (both DBs)
Broken seed / re-seed collision	seed step (both DBs)

Change	Failure surface
Any pytest regression	`backend-sqlite` pytest
`/health` or auth drift	`scripts/smoke.sh` (all envs)
Cross-org / state-machine regression	smoke + `backend-postgres` transition
Postgres-only SQL dialect break	`backend-postgres`
Image won't build or boot	`docker-build`
Docs pipeline regression	`docs` job

Local verification

```
make verify        # SQLite end-to-end
make pg-verify     # Postgres end-to-end (Docker required)
make docker-build  # build the production image
make docker-up     # full API + Postgres stack (compose)
```

Individual recipes: `install	migrate	seed	test	boot	smoke	docs	reset-db	clean`.

Dev / test / CI / prod DB separation

Surface	Path
Developer local	`apps/api/chartnav.db` (gitignored)
pytest per-test	`tmp_path / "chartnav.db"` (ephemeral)
CI (sqlite)	`$RUNNER_TEMP/chartnav_ci.db` (ephemeral)
CI (postgres)	`postgres:16-alpine` service container
Local pg-verify	throwaway `chartnav-pg-verify` container on port 55432
Prod compose	`chartnav_pgdata` persistent volume

Smoke script (shared contract)

`apps/api/scripts/smoke.sh [BASE_URL]` — 9 curl assertions:

`/health` (code + body), `/me` (401/200), `/encounters` (401/200), cross-org lens 403, own 200, cross-org 404. Exits non-zero on first failure. Used by **every** smoke path (local `make verify`, Postgres parity, Docker image CI).

What this phase adds beyond CI

- Hardened `Dockerfile` (non-root, healthcheck).
- `apps/api/entrypoint.sh` for deterministic migrate-on-start.
- `infra/docker/docker-compose.prod.yml` for the API + Postgres stack.
- `scripts/pg_verify.sh` — reproducible Postgres parity proof.

What this phase does NOT do

- No image push / registry release.
- No Kubernetes / Terraform / cloud-specific bits.
- No TLS termination or edge reverse proxy.
- No secret store integration.

Those land in a dedicated hosted-deploy phase. This phase makes sure

the seams are in place to plug any of them in without rewrites.

Doc Artifact Pipeline

Purpose

The living docs in `docs/build/*` and `docs/diagrams/*` are the source

of truth. The final consolidated artifacts in `docs/final/` are always

rebuildable from those sources — no hand edits.

Source / output layout

```
docs/
├── build/        # human-maintained markdown — source of truth
├── diagrams/     # Mermaid sources (fenced in markdown)
└── final/
    ├── chartnav-workflow-state-machine-build.html   # generated
    └── chartnav-workflow-state-machine-build.pdf    # generated
```

Builder

File: `scripts/build_docs.py`

- Pure Python, no external deps beyond the standard library.
- Resolves paths relative to the script, so the same invocation works

locally and in CI.

- Parses a small markdown subset (headings, fenced code, tables, bullet

lists, inline `code`, **`bold`**) directly to HTML.

- Mermaid fences are passed through verbatim into `<div class="mermaid">`

blocks; the HTML imports Mermaid from a CDN and renders on load.

- Writes `docs/final/chartnav-workflow-state-machine-build.html`.
- Generates the PDF via headless Chromium (`--headless=new --print-to-pdf`).

Selection order:

1. `CHARTNAV_PDF_BROWSER` env var if set (used by CI with `chromium-browser`).

2. macOS Chrome at `/Applications/Google Chrome.app/...` .

3. Chromium in a standard app bundle.

4. `chromium-browser` , `chromium` , `google-chrome` , `chrome` on `PATH` .

- If no browser is available, falls back to `reportlab` for a plain-text

PDF so CI never silently produces a zero-byte artifact.

Run it

Locally:

```
python scripts/build_docs.py          # or:  make docs
```

In CI: the `docs` job in `.github/workflows/ci.yml` installs `chromium-browser` via apt, runs the builder with `CHARTNAV_PDF_BROWSER=chromium-browser`, and uploads the resulting HTML + PDF as a `chartnav-docs-final` artifact. The upload step is configured with `if-no-files-found: error`, so a silent generation failure fails the build.

Sections consumed (in order)

1. Executive summary (inlined in the builder)
2. `docs/build/01-current-state.md`
3. `docs/build/02-workflow-state-machine.md`
4. `docs/build/03-api-endpoints.md`
5. `docs/build/04-data-model.md`
6. `docs/build/05-build-log.md`
7. `docs/build/06-known-gaps.md`
8. `docs/build/07-auth-and-scoping.md`
9. `docs/build/08-test-strategy.md`
10. `docs/build/09-ci-and-deploy-hardening.md`
11. `docs/build/10-doc-artifact-pipeline.md`
12. `docs/diagrams/system-architecture.md`
13. `docs/diagrams/encounter-status-machine.md`
14. `docs/diagrams/er-diagram.md`
15. `docs/diagrams/api-data-flow.md`

If a source file is missing, the builder emits an inline `*missing: <path>*` marker rather than crashing — a broken diagram or an undeleted section shows up immediately in the rendered output and in the CI artifact.

Why not a heavier stack

- No MkDocs / Docusaurus / Sphinx — those require a theme + build config we'd have to maintain. We currently just need the 10-page PDF to stay fresh.
- No pandoc dependency — keeps the CI image lean and the local experience `python` + a headless browser (which most devs already have).
- If we grow beyond a single consolidated artifact (e.g. a hosted docs site), this builder is easy to retire because the sources are already plain markdown.

11 — Production-auth seam

Production-Auth Seam

State (after phase 10)

Bearer mode is no longer a placeholder — it performs real JWT validation

against a JWKS endpoint. Header mode is preserved for local/dev.

Transports

Mode	State	Behavior
`header`	implemented	Reads `X-User-Email`, resolves from `users`. Dev only.
`bearer`	implemented	Reads `Authorization: Bearer <jwt>`, validates signature/iss/aud/exp against `CHARTNAV_JWT_JWKS_URL`, maps to a `users` row by `CHARTNAV_JWT_USER_CLAIM` (default `email`).

Both produce the same `Caller(user_id, email, full_name, role, organization_id)`.

JWT validation specifics

Implemented in `apps/api/app/auth.py::resolve_caller_from_bearer`:

1. Parse `Authorization: Bearer <token>`. Missing → 401 `missing_auth_header`. Malformed → 401 `invalid_authorization_header`.

2. Look up the signing key via `PyJWKClient(settings.jwt_jwks_url)` (cached). Failure → 401 `invalid_token`.

3. `jwt.decode(...)` with:

- Algorithms: `RS256 / RS384 / RS512 / ES256 / ES384 / ES512`
- `issuer=settings.jwt_issuer`
- `audience=settings.jwt_audience`
- `options={"require": ["exp", "iss", "aud"]}`

4. Specific error codes per failure:

- `token_expired` — `exp` in the past
- `invalid_issuer` — `iss` mismatch
- `invalid_audience` — `aud` mismatch
- `invalid_token` — any other JWT error

5. Pull `settings.jwt_user_claim` (default `email`) out of the claims. Missing / non-string → 401 `missing_user_claim`.

6. `fetch_one` against `users`. Missing → 401 `unknown_user`.

No fallbacks. No "if token looks roughly ok, let it through". Fail closed.

Claim mapping

`CHARTNAV_JWT_USER_CLAIM` (default `email`) names the claim whose value

is used to look up the `users.email` row. In most OIDC setups that's

exactly `email`; for deployments that use a different claim shape, this

is the single knob to turn.

The internal `users` table remains the authoritative source of

`organization_id` and `role`. Tokens carry identity, not authorization.

Test hook

`apps/api/app/auth.py::set_jwk_client(client)` lets tests inject a fake JWKS client. The real `PyJWKClient` is used in production. This is the only crack in the wall that tests are allowed to poke.

Bearer-mode test coverage (`tests/test_auth_modes.py`)

Scenario	Result
Bearer mode without JWT env → `RuntimeError` at import	✅
Header mode default contract unchanged	✅
Valid RS256 token + known user → 200 with role/org	✅
Missing Authorization → 401 `missing_auth_header`	✅
Non-Bearer scheme → 401 `invalid_authorization_header`	✅
Garbage token → 401 `invalid_token`	✅
Wrong issuer → 401 `invalid_issuer`	✅
Wrong audience → 401 `invalid_audience`	✅
Expired token → 401 `token_expired`	✅
Valid token, claim points at unknown user → 401 `unknown_user`	✅
Valid token missing the configured claim → 401 `missing_user_claim`	✅

Tests generate a local RSA keypair and sign tokens with it, then inject a lightweight JWKS stub via `set_jwk_client(...)` so no network calls happen.

What still isn't covered in this phase

- No JWKS rotation test (we cache aggressively via PyJWKClient).
- No HS256 / shared-secret flow — intentionally; production should be asymmetric.
- No refresh-token dance. ChartNav only validates access tokens.
- No revocation list. If a token needs to be killed before `exp`, the identity provider has to roll the signing key (or the user's `users` row can be deleted, which takes the caller down one hop later).

12 — Runtime config

Runtime Configuration

Single source of truth: `apps/api/app/config.py` (`settings`). Every

other module imports from there; nothing else reads `os.environ` .

The contract

Variable	Required?	Default	Purpose
`CHARTNAV_ENV`	no	`dev`	Informationa label.
`DATABASE_URL`	no (dev default)	`sqlite:///apps/api/chartnav.db`	SQLAlchemy URL. SQLite Postgres.
`CHARTNAV_AUTH_MODE`	no	`header`	`header` (de or `bearer` (prod, real JWT).
`CHARTNAV_JWT_ISSUER`	**yes iff** `AUTH_MODE=bearer`	—	OIDC issuer URL.
`CHARTNAV_JWT_AUDIENCE`	**yes iff** `AUTH_MODE=bearer`	—	Expected `au` claim.
`CHARTNAV_JWT_JWKS_URL`	**yes iff** `AUTH_MODE=bearer`	—	JWKS endpo for signing-k lookup.
`CHARTNAV_JWT_USER_CLAIM`	no	`email`	Token claim used to map `users.emai`
`CHARTNAV_CORS_ALLOW_ORIGINS`	no	`http://localhost:5173,127.0.0.1:5173,localhost:5174,127.0.0.1:5174`	CSV. Empty string ⇒ sam origin only.
`CHARTNAV_RATE_LIMIT_PER_MINUTE`	no	`120`	Per-process sliding windo on authed paths. `0` disables.
`CHARTNAV_RUN_SEED`	no	`0`	Entrypoint: r seed after migrations. Keep `0` in prod.
`API_HOST` / `API_PORT`	no	`0.0.0.0` / `8000`	Uvicorn bind

Validation on startup

`app.config._load()` is evaluated at import time. It enforces:

- `CHARTNAV_AUTH_MODE ∈ {header, bearer}` .
- If `AUTH_MODE=bearer` , **all three** JWT vars must be set — otherwise

a `RuntimeError` is raised listing the missing names. The app will

not boot half-configured.

Dev / CI / Prod profiles

Profile	How config is supplied	DB	Auth mode	Seed
Dev	`apps/api/.env.example` → `.env` ; Makefile targets	`sqlite:///apps/api/chartnav.db`	`header`	ad-hoc (`make seed`)
Tests	`apps/api/tests/conftest.py` sets env per-test	per-test SQLite in `tmp_path`	`header`	via `scripts_seed.main()`
CI	`.github/workflows/ci.yml` inline env + services	SQLite + live Postgres service	`header`	seed + re-seed in workflow
Prod	platform env / secret store	`postgresql+psycopg://...` via `infra/docker/docker-compose.prod.yml`	`bearer` (when JWT validation is wired)	off by default

Example files

- `apps/api/.env.example` — full dev contract with comments.
- `infra/docker/docker-compose.prod.yml` — wires the API container to

a Postgres service using these env vars; each has a documented

default so the file is runnable as-is for local parity tests.

Database URL specifics

- **SQLite:** `sqlite:///relative/or/absolute/path.db` (three slashes +

absolute path, or three slashes + relative path).

- **Postgres:** `postgresql+psycopg://user:pass@host:5432/dbname` .

The `+psycopg` driver selector is important — SQLAlchemy defaults to

`psycopg2` which is not installed. `psycopg[binary]` is declared in

`apps/api/pyproject.toml` as the `[postgres]` extra.

Staging env contract (phase 11)

`infra/docker/.env.staging.example` captures the contract for the

staging compose file. Every critical value is `${VAR:?}` -guarded in

`infra/docker/docker-compose.staging.yml` so missing env blocks

startup loudly. Extra staging-only vars:

Variable	Purpose
`CHARTNAV_IMAGE_OWNER`	GHCR namespace (e.g. `thekidd2227`).
`CHARTNAV_IMAGE_TAG`	Pinned image tag. Rollback = change this + restart.
`POSTGRES_DB` / `USER` / `PASSWORD` / `PORT`	Required. Never commit real values.

Dev continues to use `apps/api/.env.example` . Runbook: `21-staging-runbook.md` .

Frontend runtime config

The web app reads exactly one env var at build time:

Variable	Required?	Default	Purpose
`VITE_API_URL`	no	`http://localhost:8000`	Base URL the API client points at.

Template: `apps/web/.env.example`. Copy to `.env` to override for a non-default backend.

Vite substitutes `import.meta.env.VITE_API_URL` into the bundle at build.

Nothing else about the UI comes from env today — dev identity is selected inside the app and stored in `localStorage`.

What this phase explicitly does NOT do

- No secret store integration (AWS SM, Vault, etc.).
- No `.env` auto-loading in the API (docker-compose and operators load env).
- No per-request feature flags / remote config.

Those are safe to layer on top of this contract without reshaping it.

Deploy Target

Chosen path

Docker image built from `apps/api/Dockerfile`, wired to a Postgres service via `infra/docker/docker-compose.prod.yml`. That's the single supported deploy surface in this phase — no Kubernetes, no Terraform, no cloud-specific glue yet.

Hardened Dockerfile

`apps/api/Dockerfile`:

- `python:3.11-slim` base.
- `PYTHONDONTWRITEBYTECODE`, `PYTHONUNBUFFERED`, disabled pip cache.
- Installs `curl` only (for the container healthcheck).
- Installs the API with the `[postgres]` extra (`psycopg[binary]`).
- Copies `app/`, `alembic/`, `alembic.ini`, `scripts/`, `scripts_seed.py`, `entrypoint.sh`.
- Creates a non-root `chartnav` user (uid 1001) and runs as that user.
- `HEALTHCHECK` hits `/health` every 15s with 3 retries.
- `ENTRYPOINT` is `./entrypoint.sh`; `CMD` is the uvicorn invocation.

Entrypoint

`apps/api/entrypoint.sh`:

1. Asserts `DATABASE_URL` is set (fails loudly otherwise).
2. `alembic upgrade head` — migrations always run at start-up.
3. If `CHARTNAV_RUN_SEED=1`, runs `python scripts_seed.py` (idempotent; off by default in prod).
4. `exec "$@"` — hands off to `CMD` so uvicorn becomes PID 1's child.

Migration-on-start is the right default for this app today. It is idempotent, small, and the seed step is gated off in production.

docker-compose.prod.yml

`infra/docker/docker-compose.prod.yml` brings up two services:

- `db` — `postgres:16-alpine` with a persistent volume, healthcheck, and dev-safe default credentials (operators must override for real deploys).
- `api` — built from `apps/api/Dockerfile`, depends on `db` with `condition: service_healthy`, reads all runtime config from env.

All sensitive values (`POSTGRES_PASSWORD`, JWT settings, etc.) are `${VAR:-default}` so the file is runnable locally for parity tests but ships nothing secret.

Startup path

```
# clone + cd
cd infra/docker
# optional: override defaults via env
docker compose -f docker-compose.prod.yml up --build
```

Boot sequence per container:

1. Postgres starts, becomes healthy (~5s).

2. API container starts, entrypoint runs migrations.

3. uvicorn binds 0.0.0.0:8000.

4. HEALTHCHECK turns green.

Runtime config path

Every variable is documented in `12-runtime-config.md`. The

image respects only those — there's no bespoke `config.yaml`,

Consul, etc. to reason about.

CI build verification

`.github/workflows/ci.yml` → `docker-build` job:

1. Buildx sets up.

2. Builds `chartnav-api:ci` from `apps/api/`.

3. Runs the container with `DATABASE_URL=sqlite:///./chartnav.db` and

`CHARTNAV_RUN_SEED=1`.

4. Polls `/health`.

5. Runs the same `scripts/smoke.sh` the backend-sqlite job uses.

6. Tears down.

If the image can't boot, serve `/health`, or pass smoke, the job fails.

Staging deployment (phase 11 overlay)

Staging rides on the same Dockerfile + entrypoint. The dedicated

surface lives in `infra/docker/docker-compose.staging.yml` +

`infra/docker/.env.staging.example`, driven by

`scripts/staging_{up,verify,rollback}.sh`. Full reference:

`19-staging-deployment.md` and the operator runbook

`21-staging-runbook.md`.

Key differences from this phase's generic prod compose:

- Image is **pinned** to `CHARTNAV_IMAGE_TAG` — rollback = repin + restart.
- API healthcheck uses `/ready` (DB-aware) rather than `/health`.
- Ports bind to `127.0.0.1` only; a reverse proxy is expected to front it.
- Every critical env var is `${VAR:?msg}`-guarded so a missing value

blocks startup with an explicit error.

Out of scope this phase

- Image registry push (no target credentials by default).
- Kubernetes manifests, Helm charts, Terraform.
- TLS termination — expected to be handled by a reverse proxy in front.
- Secret management integration (AWS SM, Vault, etc.).
- Blue/green or rolling deployment orchestration.

Those are the next "hosted deploy" phase's job. The Docker seam is

real and ready to be plugged into any of them.

Postgres Parity

Why this phase

The app was born on SQLite. This phase proves the same code runs on

Postgres — migrations, seed, auth, scoping, RBAC, state machine.

Code-level changes that made this possible

The DB layer was rewritten to use **SQLAlchemy Core** (`apps/api/app/db.py`).

That single switch buys cross-dialect behavior:

- All SQL uses **named** bind parameters (`:name`). No more `?` .
- Portable functions only — `COALESCE` replaced `IFNULL` ; `CURRENT_TIMESTAMP` already worked on both.
- `insert_returning_id(conn, table, values)` helper encapsulates the

dialect split: `RETURNING id` on Postgres, `cursor.lastrowid` on

SQLite. Callers never touch either.

- `transaction()` context manager gives a SA connection with `begin()`

semantics on either backend.

Consumers changed:

- `apps/api/app/api/routes.py` — all queries now use `:name` binds

and `transaction()` .

- `apps/api/scripts_seed.py` — rewritten to `transaction()` +

`insert_returning_id` , no more raw `sqlite3` .

- `apps/api/app/auth.py` — uses `fetch_one` from the new `db` module.
- `apps/api/alembic/env.py` — honors both `-x sqlalchemy.url=` and

`DATABASE_URL` env, so migrations can run against either backend.

No existing migration (`43ccbf363a8f` , `a1b2c3d4e5f6`) needed changes.

They already used SQLAlchemy constructs and portable defaults.

Proof — `scripts/pg_verify.sh`

Single reproducible command. Launches `postgres:16-alpine` in a

throwaway container, runs migrations + seed, boots the API against it,

runs the full smoke, then does a live **status transition** + reads

the resulting `workflow_events` row back.

Local run on 2026-04-18:

```
==> alembic upgrade head on Postgres
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Running upgrade  -> 43ccbf363a8f, create initial chartnav tables
INFO  [alembic.runtime.migration] Running upgrade 43ccbf363a8f -> a1b2c3d4e5f6, add encounters and workflow_events

==> seed (idempotent, run twice)
Seed complete.
```

```
  demo-eye-clinic: organization_id=1 location_id=1
  northside-retina: organization_id=2 location_id=2

==> smoke: http://127.0.0.1:8766
  ok  /health 200
  ok  /health body.status=ok
  ok  /me 401 without auth
  ok  /me 200 role=admin org=1
  ok  /encounters 401 without auth
  ok  /encounters 200 admin1
  ok  /encounters?organization_id=2 (cross-org) -> 403
  ok  GET /encounters/1 as admin1 -> 200
  ok  GET /encounters/3 cross-org -> 404
==> all smoke checks passed
==> extra: status transition on Postgres
  ok  PT-1001 in_progress -> draft_ready (clinician)
  ok  workflow_events row recorded on Postgres
==> Postgres parity: PASS
```

CI coverage

`.github/workflows/ci.yml` → `backend-postgres` job:

1. Spins up a `postgres:16-alpine` service container on port 5432.

2. Installs the API with the `[postgres]` extra.

3. `alembic upgrade head` (direct env `DATABASE_URL=postgresql+psycopg://…`).

4. Seeds twice (idempotency proof).

5. Boots uvicorn, waits for `/health` .

6. Runs `scripts/smoke.sh` .

7. Executes the same `clinician → draft_ready` status transition and

asserts the returned row.

If any step fails, the job fails and nothing merges.

What's covered / not covered

Surface	Verified on Postgres
Schema migrations	✅
Idempotent seed	✅
App boot	✅
`/health` , `/me`	✅
`/encounters` (list + filters + cross-org 403)	✅
`/encounters/{id}` 200 / 404 cross-org	✅
State transition write + event emit	✅
pytest suite (28 tests)	Runs on SQLite only today — see Known Gaps.

The pytest suite still targets SQLite because the TestClient fixture

uses `tmp_path` file DBs. Migrating that to a Postgres matrix is the

obvious next CI upgrade; the DB layer is ready for it.

Frontend Integration

Goal

Turn the backend workflow slice into a usable app. An operator should

be able to identify themselves, filter their org's encounters, open one,

see its timeline, append events, and drive role-appropriate status

transitions — all through the UI — without the backend contract changing.

Stack

- Vite 5 + React 18 + TypeScript (already in the repo).
- No component library yet — one small CSS file (`styles.css`) + inline

attributes. Easy to swap later if the UI grows.

- No state library — `useState` / `useEffect` around a typed API module.

Source layout

```
apps/web/src/
├── api.ts            # typed API client (single fetch surface)
├── identity.ts       # dev identity helpers (seeded list + localStorage)
├── App.tsx           # shell, list, detail, timeline, actions
├── main.tsx          # React entrypoint, imports styles.css
├── styles.css        # one CSS file (CSS variables; no build config)
└── vite-env.d.ts     # Vite ambient types
```

`api.ts` — contract

- `API_URL` comes from `VITE_API_URL` (falls back to `http://localhost:8000`).
- Every function takes an `email: string` ; the client sets

`X-User-Email` on each request. When the backend moves to bearer

mode, only this module changes.

- `ApiError(status, errorCode, reason)` — every non-2xx response is

converted to this, so UI code can render the backend's

`{error_code, reason}` envelope directly.

- Exposed functions (match the backend 1:1):
- `getHealth()`
- `getMe(email)`
- `listEncounters(email, filters)`
- `getEncounter(email, id)`
- `getEncounterEvents(email, id)`
- `createEncounterEvent(email, id, {event_type, event_data})`
- `updateEncounterStatus(email, id, status)`
- `listLocations(email)`
- Pure helpers used for UI affordances only:
- `allowedNextStatuses(role, currentStatus)` — mirrors

`authz.TRANSITION_ROLES`. Drives which transition buttons appear.

- `canCreateEvent(role)` — drives the "Add event" composer.
- **Backend remains the source of truth** — the UI never assumes the server will accept anything; any 4xx surfaces as a banner with the error code + reason.

Dev identity

- `apps/web/src/identity.ts` ships the seeded demo users:
- Org 1: `admin@`, `clin@`, `rev@chartnav.local`
- Org 2: `admin@`, `clin@northside.local`
- Selector in the header lets operators switch caller without editing code. "Custom email…" lets a real/unseeded email be entered for testing 401 paths.
- Selection persists in `localStorage.chartnav.devIdentity`.

UI layers

Header

- Brand wordmark.
- Chip showing the resolved caller (`email · role · org N`) — or the auth error if `/me` failed.
- Chip with the current API base URL.
- Identity picker.

Layout (two-column, collapses below 920px)

- **Encounter list (left)** — filter bar (`status`, `provider_name`, `location_id`) + row list with patient id, name, provider, and a color-coded status pill.
- **Detail pane (right)** — headline + status pill, facts grid (organization / location / scheduled / started / completed / created), allowed-transition buttons, timeline of events, and "Add event" composer when the current role is permitted to write.

Role-aware actions

- Allowed transitions are shown as buttons. If none are available for the (role, current_status) pair, we show a note instead of fake disabled buttons.
- The "Add event" composer is hidden for reviewers (matches `CAN_CREATE_EVENT`). Reviewers still get a subtle note explaining why.
- After any successful action the UI refreshes: detail, events, and list.
- After a failure, the UI shows the exact `error_code` + `reason` from the backend in a top banner.

UX states

State	Surface
Loading	"Loading…" in list / detail
Empty list	"No encounters match these filters."
401 / unknown	Red chip in the header: `auth: 401 unknown_user — ...`
403 role	Banner: `403 role_cannot_transition — ...`
404 cross-org	Banner: `404 encounter_not_found — ...`
400 bad status	Banner: `400 invalid_transition — ... allowed next ...`

Local dev

```
# one terminal — backend
make install          # venv + pip install -e "apps/api[dev,postgres]"
make reset-db         # fresh SQLite + seed
make boot             # uvicorn on 8000

# second terminal — frontend
make web-install      # npm install
make web-dev          # vite on 5173

# or both at once (Ctrl-C to stop both)
make dev
```

`VITE_API_URL` in `apps/web/.env.example` defaults to

`http://localhost:8000` , matching `make boot` . Copy to `.env` if you

need to point at a different backend.

Build / verification

```
make web-typecheck    # tsc --noEmit
make web-build        # vite build -> apps/web/dist
```

Verified manually on 2026-04-18:

Check	Result
`tsc --noEmit`	✅
`npm run build` emits dist/	✅ (154 KB JS / 6 KB CSS)
`GET /me` for all 5 seeded users	✅
`GET /encounters` org-scoped (org1 → 2 rows; org2 → `[3]`)	✅
`GET /encounters?status=in_progress`	✅ `['PT-1001']`
`GET /encounters/{id}/events` hydrated	✅ 3 events
Clinician in_progress → draft_ready	✅ 200
Clinician review_needed → completed (denied)	✅ 403 `role_cannot_transition`
Reviewer review_needed → completed	✅ 200
Reviewer POST event (denied)	✅ 403 `role_cannot_create_event`
Admin POST event	✅ 201
Switching identity re-fetches `/me` and list	✅

Encounter creation (phase 8)

- Header shows `+ New encounter` for admin + clinician; hidden for reviewer.
- `CreateEncounterModal` fetches `/locations` (server-scoped to caller org), auto-selects when only one option exists, and submits `{ organization_id, location_id, patient_identifier, patient_name, provider_name, status }`.
- Required fields (`patient_identifier` , `provider_name` , `location_id`) block submit until filled.
- While in-flight the submit button disables and shows `Creating…` .
- On success the list refreshes, the new encounter auto-selects, and a green banner shows `#<id> created` .
- On failure the modal stays open and the exact `{error_code, reason}` appears inline, so reviewers can correct and retry.

UX hardening (phase 8)

- Every mutating control (transition, append event, create) disables while its request is in flight and shows a pending label.
- Identity badge has explicit states (`identity-loading` , `identity-error` , `identity-badge`) so loading vs. failed vs. resolved are distinguishable.
- Banners use ARIA `role="alert" | "status"` and `data-testid` so assistive tech + tests can target them.
- When no transition is legal for the current `(role, status)` pair, we show a plain note — no fake-disabled buttons.

Full-stack E2E (phase 9)

Playwright boots both backend (SQLite + seeded) and frontend together,

and runs 8 Chromium scenarios that exercise the real UI end-to-end —

identity resolution, scope switching, encounter create, event append,

role-aware transitions, reviewer restrictions, unknown-email auth

surface, filters. Details in `17-e2e-and-release.md` . Command:

`make e2e` .

What this phase explicitly does NOT do

- No real login flow — `X-User-Email` is still dev transport.
- No global state manager (Redux/Zustand) — state lives in `App.tsx` .
- No UI component library.
- No pagination (the backend doesn't paginate yet).
- No optimistic updates — every mutation re-fetches the relevant slice.

Those are future phases. The API client + identity seam make them

cheap to add without rewriting.

Frontend Test Strategy

Stack

- **Vitest** (`vitest` + `@vitest/ui`) — same Vite runtime as the app.
- **jsdom** — DOM environment.
- **@testing-library/react** + **@testing-library/user-event** — user-centric queries, realistic events.
- **@testing-library/jest-dom** — richer DOM matchers.

Layout

```
apps/web/
├── vite.config.ts            # vitest config block (jsdom, globals, setup file)
├── src/test/
│   ├── setup.ts              # imports jest-dom matchers, cleans DOM + localStorage between tests
│   └── App.test.tsx          # 12 integration tests against the mocked API layer
└── package.json              # scripts.test = vitest run, test:watch = vitest
```

Mocking strategy

`vi.mock("../api", ...)` replaces every backend call with a `vi.fn()`

while preserving the pure helpers (`allowedNextStatuses` , `ApiError` ,

etc.). That keeps tests decoupled from the network, deterministic, and

fast — but still exercises the real `App.tsx` component tree.

The mocks emulate the server's real error envelope so the UI code's

`{error_code, reason}` branch is actually tested.

Coverage (12 tests)

Scenario	Check
`/me` resolves → identity badge shows email/role/org	✅
Mocked list renders both seeded encounters	✅
Changing `status` filter calls `listEncounters` with filter arg, list updates	✅
Selecting an encounter loads detail + timeline; status shown	✅
Clinician sees only operational transition buttons (`draft_ready`)	✅
Reviewer sees review-stage transitions (`completed` , `draft_ready`)	✅
Reviewer sees no event composer, replaced by `event-denied` note	✅
Reviewer cannot see the `+ New encounter` button	✅
Admin: open create modal, submit, success banner, modal closes	✅
Create failure: backend 403 `cross_org_access_forbidden` surfaces in modal error, modal stays open	✅
Switching identity refetches `/me` and the list (org1 → org2)	✅
Unknown-user identity: custom email path surfaces the 401 chip	✅
Status transition call path refreshes detail + events, success banner	✅

Run

```
# one-shot (CI + local gate)
cd apps/web && npm test

# watch mode
cd apps/web && npm run test:watch

# combined frontend gate
make web-verify      # typecheck + test + build
```

Result at time of writing: **12 passed in ~2.5s**.

Philosophy

- The UI is tested the way a user drives it — click, select, type — not

via internal implementation details.

- We assert the UI **reflects the server's authority**: when the backend

returns a 4xx envelope, the UI must show the `error_code` and

`reason` verbatim, not invent its own copy.

- Role-aware affordances are tested by identity switching in the

picker, not by poking at internal state.

End-to-end tests (Playwright)

Full browser coverage ships in phase 9 — see `17-e2e-and-release.md` .

Playwright boots real backend + frontend, runs 8 scenarios in Chromium,

and tears both servers down cleanly. Command: `make e2e` .

Vitest and Playwright are fully isolated: Vitest's `test.include` is

narrowed to `src/**/*.test.{ts,tsx}` and excludes `tests/**` , so the

two suites never cross-contaminate.

Gaps not yet covered

- No visual regression / accessibility audits.
- The create modal doesn't exercise location-list errors yet.
- Loading skeletons / spinner content not asserted.

E2E & Release

E2E harness

Stack

- **Playwright** (`@playwright/test`) with the `chromium` browser.
- Playwright's `webServer` boots **both** backend and frontend, waits for

their `/health` / `/` URLs, and tears both down on exit. No bash

orchestration, no orphan processes.

Config

`apps/web/playwright.config.ts` :

- **Backend** boots on `127.0.0.1:8001` using an ephemeral SQLite file

at `apps/api/.e2e.chartnav.db` (the operator's dev DB is never

touched). The config runs:

```

rm -f .e2e.chartnav.db

alembic upgrade head

python scripts_seed.py

uvicorn app.main:app --port 8001

```

- **Frontend** boots on `127.0.0.1:5174` via `npm run dev -- --host 127.0.0.1 --port 5174` , with `VITE_API_URL` pointed at the E2E backend.
- `reuseExistingServer: !CI` — locally, running `make e2e` twice in a

row reuses the stack; in CI each run is fresh.

- Traces, screenshots, videos retained on failure only.

Vitest / Playwright isolation

`vite.config.ts` now narrows Vitest `test.include` to `src/**/*.test.{ts,tsx}` and excludes `tests/**` , so Playwright specs and Vitest specs never cross-contaminate.

E2E coverage

`apps/web/tests/e2e/workflow.spec.ts` (8 tests):

Scenario
App boots, resolves default seeded identity
Switching identity (admin1 → admin2) swaps visible encounter scope
Admin opens detail, creates encounter, sees it in the list
Admin appends an event; it appears in the timeline
Clinician performs operational transitions; reviewer edge not offered
Reviewer sees completion/kick-back controls, no create button, no event composer
Unknown email surfaces `identity-error` chip with `unknown_user`

Scenario
Status filter narrows the list

Tests use resilient selectors: `role`, placeholder text, and `data-testid` for elements without another stable hook.

Local run:

```
make e2e          # headless
make e2e-headed   # visible browser
make e2e-ui       # Playwright UI mode
```

Result at time of writing: **8/8 passed in ~14s**.

CI

New `e2e` job in `.github/workflows/ci.yml`:

1. `backend-sqlite + frontend` must pass first.

2. Set up Python 3.11 + Node 20 (both with caches).

3. `pip install -e "apps/api[dev,postgres]"` (system install — verify.sh already tolerates missing `.venv`).

4. `npm ci` in `apps/web`.

5. `npx playwright install --with-deps chromium`.

6. `npx playwright test --reporter=list` (Playwright boots + tears down the stack).

7. On failure, upload `playwright-report/ + test-results/` as an artifact for debugging.

Release pipeline

`scripts/release_build.sh`

One command to produce a releasable bundle under `dist/release/<version>/`:

- `chartnav-api-<version>.tar` — `docker save` of `chartnav-api:<version>` (importable with `docker load`).
- `chartnav-web-<version>.tar.gz` — static bundle from `apps/web/dist` (drop behind any CDN/reverse proxy).
- `MANIFEST.txt` — version, git sha, ref, build time, artifact sizes + sha256s.

Version resolution:

1. Explicit arg (e.g. `bash scripts/release_build.sh v0.1.0`).

2. Exact git tag on `HEAD`.

3. Fallback `dev-<short-sha>`.

Local:

```
make release-build VERSION=v0.1.0
```

`dist/release/` is gitignored.

`.github/workflows/release.yml`

Triggers:

- `push` on tags matching `v*.*.*` (e.g. `v0.1.0`, `v1.2.3-rc1`).
- `workflow_dispatch` with a `version` input (manual).

Steps:

1. Resolve version from the tag or input.

2. Install `apps/web` deps (Node 20).

3. Login to `ghcr.io`.

4. `docker buildx build + push` of `ghcr.io/<owner>/chartnav-api:<version>` **and** `:latest`.

5. Run `scripts/release_build.sh <version>` to produce the local artifact set.

6. Upload the whole `dist/release/<version>/` as a workflow artifact (always).

7. On tag pushes only, create a **GitHub Release** with:

- auto-generated notes from the commit log
- `chartnav-api-<version>.tar`, `chartnav-web-<version>.tar.gz`, `MANIFEST.txt` attached.

Permissions: `contents: write` for the release, `packages: write` for `ghcr.io`.

Release is never auto-triggered from CI commits — only an explicit tag push or manual dispatch cuts one.

Release flow (operator view)

```
# 1. work lands on main, CI green
git checkout main && git pull --ff-only

# 2. tag
git tag -a v0.1.0 -m "ChartNav v0.1.0"
git push origin v0.1.0

# 3. GitHub Actions runs the release workflow, pushes the GHCR image,
#    creates the Release, attaches the tarballs.

# 4. deploy
docker pull ghcr.io/<owner>/chartnav-api:v0.1.0
docker compose -f infra/docker/docker-compose.prod.yml up -d
```

Staging artifact (phase 11)

`scripts/release_build.sh` now also writes `chartnav-staging-<version>.tar.gz`

containing the staging compose file, its env template, and the runbook

scripts (`staging_up.sh`, `staging_verify.sh`, `staging_rollback.sh`)

plus the three new doc pages (19/20/21). `release.yml` attaches the

tarball to the GitHub Release alongside the API image tar and web

bundle, so a staging operator can `curl` one archive and be ready to

run against the pinned image tag.

What this phase does NOT do

- No staging / production deploy target is connected — the release workflow builds and publishes, operators still run `docker compose` wherever they host it.
- No automated Postgres E2E — Playwright's webServer uses SQLite for speed + isolation. Postgres parity is still proven by `backend-postgres`.
- No rollback automation — rollbacks are `docker pull` of the previous tag.
- No signing / SBOM yet. Those are safe to add on top of this pipeline.

18 — Operational hardening

Operational Hardening

Landing this phase gives ChartNav four production-baseline primitives

that every long-running service needs: **request correlation**,

structured logs, **audit trail for denied access**, and **abuse

protection**.

1. Request correlation

Middleware: `apps/api/app/middleware.py::RequestIdMiddleware`.

- Every request gets a `request_id` on `request.state`.
- Honors an inbound `X-Request-ID` if it's a reasonable length (≤ 64 chars); else generates a fresh UUID4 hex.
- Always echoed back as `X-Request-ID` on the response — including error responses (the exception handler copies it through).

2. Structured logging

Module: `apps/api/app/logging_config.py`.

- JSON-per-line to stdout via `JsonFormatter`.
- Fields: `timestamp` (UTC ISO8601), `level`, `logger`, `message`, plus any `extra={}` the caller passes.
- `AccessLogMiddleware` emits one line per request with `request_id`, `method`, `path`, `status`, `duration_ms`, `user_email` (from resolved caller), `organization_id`, `remote_addr`.
- The HTTP exception handler emits a `WARNING auth_denied` line for every denied auth/scoping response, attaching `error_code` and caller context.
- Uvicorn's access logger is quieted to avoid duplicating our access log.

Log sample (pretty-printed here for readability):

```
{"timestamp":"2026-04-18T05:35:45Z","level":"INFO","logger":"chartnav.http",
 "message":"request","request_id":"494e2888ab414831805bf79cdefa7fc2",
 "method":"GET","path":"/me","status":401,"duration_ms":3.55,
 "user_email":null,"organization_id":null,"remote_addr":"testclient"}
```

3. Security audit trail

Module: `apps/api/app/audit.py`.

Migration: `b2c3d4e5f6a7 — add security_audit_events`.

Schema

Table `security_audit_events`:

- `id`, `event_type`, `request_id`, `actor_email`, `actor_user_id`, `organization_id`, `path`, `method`, `error_code`, `detail`, `remote_addr`, `created_at`.
- Indexed on `event_type`, `actor_email`, `created_at`.

What gets audited

The HTTP exception handler (`apps/api/app/main.py::_http_exception_handler`) writes an audit row when:

- The response is **401** or **403**, or
- The response `error_code` is in the `AUDITED_ERROR_CODES` set:

`missing_auth_header`, `unknown_user`, `invalid_authorization_header`, `invalid_token`, `token_expired`, `invalid_issuer`, `invalid_audience`, `missing_user_claim`, `cross_org_access_forbidden`, `role_forbidden`, `role_cannot_create_encounter`, `role_cannot_create_event`, `role_cannot_transition`, `rate_limited`.

What is NOT audited

- No raw JWT or `Authorization` header value.
- No password material (there isn't any, but explicitly).
- No successful requests — those belong in the access log.

Never-throw guarantee

`audit.record(...)` swallows its own exceptions. A broken audit insert cannot mask the underlying 4xx the user actually hit.

4. CORS

- `allow_origins=["*"]` is gone.
- Driven by `CHARTNAV_CORS_ALLOW_ORIGINS` (comma-separated). Default covers Vite dev on `:5173` + E2E on `:5174`.
- In production, set it explicitly to the real frontend origin(s).
- Empty string → same-origin only.
- `allow_credentials=true`; `allow_headers` is limited to `Authorization`, `Content-Type`, `X-User-Email`, `X-Request-ID`; `expose_headers` exposes `X-Request-ID` so browsers can surface it for debugging.

5. Rate limiting

Middleware: `apps/api/app/middleware.py::RateLimitMiddleware`.

- In-memory sliding window (60s), per-process.
- Keyed on `(client_ip, path)`.
- `CHARTNAV_RATE_LIMIT_PER_MINUTE` (default 120). `0` disables.
- Protects `/me`, `/encounters`, `/organizations`, `/locations`, `/users` (and descendants). `/health` and `/` are never rate-limited.
- On limit exceed: `HTTP 429` with standardized envelope `{error_code: "rate_limited", reason: ...}` AND an audit row.
- Honest limitation: **per-process**, not distributed. Multi-worker / multi-node deployments should sit behind an edge limiter that coordinates globally. Documented in `12-runtime-config.md`.

Config reference

Env var	Default
`CHARTNAV_CORS_ALLOW_ORIGINS`	`http://localhost:5173,http://127.0.0.1:5173,http://localhost:5174,http://127.0.0.1:5174`
`CHARTNAV_RATE_LIMIT_PER_MINUTE`	`120`

See `12-runtime-config.md` for the full contract.

Test coverage

`apps/api/tests/test_operational.py` (12 tests):

Scenario	Result
Inbound `X-Request-ID` round-trips	✅
Missing `X-Request-ID` → server generates one	✅
Error responses also carry `X-Request-ID`	✅
Audit row on `missing_auth_header`	✅
Audit row on `unknown_user`	✅
Audit row on `cross_org_access_forbidden` (query lens)	✅

Scenario	Result
Audit row on `role_cannot_create_encounter`	✅
No audit row on successful requests	✅
Rate limit returns **`429`** **`rate_limited`** with envelope	✅
Rate limit disabled when `CHARTNAV_RATE_LIMIT_PER_MINUTE=0`	✅
CORS preflight allows a configured origin	✅
CORS preflight rejects an unconfigured origin	✅

Observability (phase 11 additions)

This baseline now includes a live observability surface — see

`20-observability.md` :

- `GET /ready` — DB-aware readiness. Backs the staging container `HEALTHCHECK` .
- `GET /metrics` — Prometheus text exposition. Counters cover request traffic, auth denials (by `error_code`), rate-limited responses, audit event writes (by `event_type`), and request latency sum/count.
- Middleware + exception handler now feed those counters as part of normal request handling. `audit.record` also bumps `chartnav_audit_events_total{event_type}` before the DB insert.

Remaining gaps

- Rate limiter is in-memory per process — same caveat applies to the new metrics.
- Distributed tracing (OpenTelemetry) not wired yet.
- No log shipping / retention policy defined.
- Audit table has no retention/archival yet.
- No dashboards/alerts are shipped — `20-observability.md` documents the alert points, wiring is the operator's responsibility.

Staging Deployment

Shape

ChartNav's staging deploy unit is a **pinned Docker image** (`ghcr.io/<owner>/chartnav-api:<tag>`) plus a `postgres:16-alpine` container, wired together by `infra/docker/docker-compose.staging.yml` . That's the whole substrate. There is no Kubernetes, no Terraform, no serverless. Rolling out = change a tag + restart. Rolling back = change the tag back + restart.

This is intentional. ChartNav is still a small service; a boring, reproducible compose deploy is the honest size. Phase-out-to-something-bigger will be a deliberate future phase, not an accident.

Artifacts added this phase

Path	Purpose
`infra/docker/docker-compose.staging.yml`	API + Postgres, pinned image, healthcheck on `/ready` , volumes, restart policy.
`infra/docker/.env.staging.example`	The env contract. Copy to `.env.staging` and fill in.
`scripts/staging_up.sh`	`docker compose config` -validate then `up -d` .
`scripts/staging_verify.sh`	Smoke + observability check against the running stack.
`scripts/staging_rollback.sh <prev_tag>`	Repin `CHARTNAV_IMAGE_TAG` , pull, restart API, wait for `/ready` .

Topology

```
┌─────────────────┐        ┌──────────────────────┐
│  reverse proxy  │──TLS──▶ api:8000 (127.0.0.1)  │
│  (ops layer)    │        │ ghcr.io/..../chartnav │
└─────────────────┘        │ -api:${TAG}           │
                           │ entrypoint: migrate   │
                           │ + uvicorn             │
                           └──────────┬────────────┘
                                      │ postgresql+psycopg
                                      ▼
                            ┌──────────────────┐
                            │ db:5432          │
                            │ postgres:16-alpine │
                            │ volume:          │
                            │  chartnav_staging_ │
                            │  pgdata          │
                            └──────────────────┘
```

- API and DB both bind to `127.0.0.1` on the host — reverse proxy terminates TLS and handles public routing.
- Healthcheck runs `curl -fsS http://127.0.0.1:8000/ready` inside the container: because `/ready` touches the DB, that one call also verifies migrations ran and the DB connection is live.
- DB persistence: named volume `chartnav_staging_pgdata` . Volumes survive `docker compose down` ; they do **not** survive `docker compose down -v` . A rollback to a prior image tag never needs to touch the volume.

Migration-on-start

`apps/api/entrypoint.sh` runs `alembic upgrade head` before it execs `uvicorn` . Staging respects that, so "deploy = new image" and "migrate" are the same action. Two properties fall out for free:

- Deploys are idempotent: apply the same tag twice, get the same schema.
- Downgrade paths are an image concern, not an ops concern: redeploying an older image with its own Alembic head will only try to apply migrations it knows about.

If a migration ever needs to run offline (big data reshape), that's the one thing this flow doesn't help with — pin the old image, take an outage window, run `alembic` manually, then deploy the new image.

Required env contract (summary)

Full reference: `12-runtime-config.md`. Staging adds nothing new; it just requires real values for what was optional in dev:

Var	Notes
`CHARTNAV_IMAGE_OWNER` / `_TAG`	GHCR owner + pinned tag.
`POSTGRES_DB/USER/PASSWORD`	Never commit real values.
`CHARTNAV_AUTH_MODE`	`bearer` in real staging. `header` only for first-touch QA.
`CHARTNAV_JWT_*`	Required when `AUTH_MODE=bearer`; the app refuses to import otherwise.
`CHARTNAV_CORS_ALLOW_ORIGINS`	**Never `*`**. Real staging frontend origin(s) only.
`CHARTNAV_RATE_LIMIT_PER_MINUTE`	Defaults to 120; bump for load-test windows.

Compose uses `${VAR:?message}` for the critical vars so a missing value blocks `docker compose up` with an explicit error.

Commands

```
# first-time: copy + fill in
cp infra/docker/.env.staging.example infra/docker/.env.staging
# edit infra/docker/.env.staging with real values

# deploy
make staging-up          # docker compose config + up -d
make staging-verify      # smoke + observability checks (see 21)
# real traffic

# rollback to a previously-published tag
make staging-rollback TAG=v0.0.9
make staging-verify

# teardown (keeps the DB volume)
make staging-down
# nuke DB too
cd infra/docker && docker compose -f docker-compose.staging.yml --env-file .env.staging down -v
```

Rollback — honest description

"Rollback" in this deploy is:

1. `scripts/staging_rollback.sh <prev_tag>` rewrites `CHARTNAV_IMAGE_TAG` in `.env.staging`.
2. `docker compose pull api` fetches the prior image from GHCR.
3. `docker compose up -d api` restarts only the API service; Postgres is untouched.
4. The script polls `/ready` and fails loudly if it doesn't come back green within ~40s.

What rollback **cannot** do on this substrate:

- Undo a destructive migration. If revision N dropped a column, going back to N-1's image won't restore the column. Treat destructive migrations as one-way on this deploy.
- Blue/green. There's one API replica.
- In-flight request draining. `docker compose up -d` is SIGTERM-then-wait. If you need graceful draining of long operations, put them behind a queue, not the web process.

Those are fair tradeoffs for a compose-based staging. The migration-on-start path also means the rollback script doesn't need to run Alembic by hand — the old image knows its own head.

CI validation

`.github/workflows/ci.yml` → `deploy-config` job: on every push/PR it runs `docker compose … config` on all three compose files (dev/staging/prod) and `shellcheck` on every repo script. A broken compose file or sloppy script lands a failing check before the operator ever tries to `staging-up`.

Observability

ChartNav's observability baseline is honest: three surfaces (`/health` , `/ready` , `/metrics`), one structured log stream, and one audit table. No platform fantasy.

Surfaces

`GET /health` — liveness

Cheap. Never touches the DB. Use it for load-balancer keepalives where "the process is answering" is enough.

```
200 {"status":"ok"}
```

`GET /ready` — readiness

Runs `SELECT 1` against the database. Use it for orchestrator healthchecks where you want to gate traffic on DB reachability. `503 not_ready` if the DB is unreachable.

```
200 {"status":"ready","database":"ok"}
```

In `infra/docker/docker-compose.staging.yml` , the API container's `HEALTHCHECK` targets `/ready` — not `/health` — because a running uvicorn with a broken DB connection is worse than no uvicorn.

`GET /metrics` — Prometheus text exposition

Unauthed. Exposed on the internal network only; restrict at the edge if you must expose publicly (see below).

Counters emitted:

Metric	What it answers
`chartnav_requests_total{method,path,status}`	Traffic by (method, path, status bucket `1xx..5xx`).
`chartnav_auth_denied_total{error_code}`	Denied auth/scoping attempts, keyed on the stable error codes from the `{error_code, reason}` envelope.
`chartnav_rate_limited_total`	`429` `rate_limited` responses.
`chartnav_audit_events_total{event_type}`	Audit rows written, by event type.
`chartnav_http_request_duration_ms_sum`	Cumulative request latency, milliseconds.
`chartnav_http_request_duration_ms_count`	Sample count matching the sum. Divide for a mean.

Honest caveats:

- **In-process, single-worker.** Same caveat as the rate limiter. Multi-worker uvicorn = each worker scrapes its own counters. A proper metrics lib backed by Prometheus multiprocess mode is the next step if that becomes a real problem.
- No histograms / percentiles. `sum + count` is the honest minimum; percentiles lie when they're synthesized from one bucket.

Structured logs

All logs are line-delimited JSON on stdout. Ship them with whatever collector the deploy target uses (compose log driver, cloudwatch agent, vector, fluent-bit — the app doesn't care).

Access log (INFO, `logger=chartnav.http`)

Emitted once per request:

```
{
  "timestamp": "2026-04-18T15:22:10Z",
  "level": "INFO",
  "logger": "chartnav.http",
  "message": "request",
  "request_id": "a1b2c3...",
  "method": "POST",
  "path": "/encounters/1/status",
  "status": 200,
  "duration_ms": 7.42,
  "user_email": "clin@chartnav.local",
  "organization_id": 1,
  "remote_addr": "10.0.0.42"
}
```

Auth-denied log (WARNING, `logger=chartnav`)

Emitted alongside every audited denial — mirror image of the audit row:

```
{
  "timestamp": "2026-04-18T15:22:10Z",
  "level": "WARNING",
  "logger": "chartnav",
  "message": "auth_denied",
  "request_id": "a1b2c3...",
  "path": "/me",
  "method": "GET",
  "status": 401,
  "error_code": "missing_auth_header",
  "user_email": null,
  "organization_id": null
}
```

Request failure log (ERROR)

Uncaught exceptions surface with a stack trace via the access-log middleware. Always carries `request_id`.

Fields guaranteed across all records

- `timestamp` — UTC ISO8601
- `level` — standard logging level
- `logger`
- `message`
- `request_id` — when the log is request-scoped

Collection notes

- Docker compose: default stdout goes to the `json-file` driver unless you override `logging.driver` on the service. For staging, point at your collector (`loki`, `fluent-bit`, etc.) via a compose override file; the app emits valid JSON already.
- Do not parse with `grep` in production. Ship to a log store and query by `request_id`, `error_code`, or `user_email`.

Audit table (`security_audit_events`)

See `18-operational-hardening.md` for the schema. The audit table is a durable peer to the logs: logs answer "what happened and when", the audit table answers "what denied-access attempts, who did them, to which org, how often". They're not redundant — logs churn, audit rows persist.

Query examples:

```
-- recent denials by error_code
SELECT error_code, COUNT(*) AS n
FROM security_audit_events
WHERE created_at > now() - interval '24 hours'
GROUP BY error_code
ORDER BY n DESC;
```

```
-- repeated cross-org attempts from one actor
SELECT actor_email, path, COUNT(*) AS n
FROM security_audit_events
WHERE event_type = 'cross_org_access_forbidden'
  AND created_at > now() - interval '7 days'
GROUP BY actor_email, path
HAVING COUNT(*) >= 3;
```

What to check during staging smoke

The one-shot is `make staging-verify` (see `21-staging-runbook.md`). Interactively, an operator should look at:

1. **Liveness**: `curl -fsS http://.../health` → `200`.

2. **Readiness**: `curl -fsS http://.../ready` → `200 database=ok`. If this fails, API logs will explain; `docker compose logs db` will confirm.

3. **Request id round-trip**: `curl -I -H "X-Request-ID: probe-$$" http://.../health` — response must carry the same id back.

4. **Auth surface**:

- `curl -i http://.../me` without auth → 401 + a `missing_auth_header` entry on the next `/metrics` scrape.
- `GET /metrics` contains `chartnav_auth_denied_total{error_code="missing_auth_header"}`.

5. **Rate-limit sanity**: quickly spam `GET /me` to exceed the configured `CHARTNAV_RATE_LIMIT_PER_MINUTE`; expect a 429 and `chartnav_rate_limited_total` to tick up.

6. **Audit row check**: `docker compose exec -T db psql -U $POSTGRES_USER -d $POSTGRES_DB -c "SELECT event_type, COUNT(*) FROM security_audit_events GROUP BY 1 ORDER BY 2 DESC;"`.

Alert points (contract, not implementation)

When an operator wires alerting, these are the knobs that carry real meaning:

Signal	Meaning
`/ready` failing for >1 minute	DB connection lost or pool exhausted.
`chartnav_auth_denied_total{error_code="invalid_token"}` burst	IdP trouble or attack traffic hitting `Authorization:`.
`chartnav_auth_denied_total{error_code="cross_org_access_forbidden"}` spike	Misconfigured client, token reuse across orgs, or probing.
`chartnav_rate_limited_total` steep climb	Abuse or a stuck client.
Audit table growth rate ≫ steady-state	Investigate; correlate with the two metrics above via request_id.

This phase does not ship dashboards or alert rules. The plumbing is in place; wire them in your observability platform of choice (Grafana/Datadog/CloudWatch/etc.).

Remaining gaps

- No tracing / span propagation yet (OpenTelemetry is the natural next layer).
- Metrics are per-process.
- No log retention or audit-table retention/archival policy in the app; that's ops infra's job.

21 — Staging runbook

Staging Runbook

Operator-grade playbook. Every command is real and reproducible. No magic.

---

0. Prerequisites

- Docker + `docker compose` v2.
- Access to `ghcr.io/<owner>/chartnav-api` (at minimum anonymous pull; the image is published public on release).
- A JWKS URL from your IdP (for bearer mode).
- Ability to place DNS / reverse-proxy config in front of port 8000 on the staging host. The compose file binds to `127.0.0.1:8000` — your reverse proxy is expected to handle public exposure and TLS.

1. First-time provisioning

```
# on the staging host
git clone https://github.com/thekidd2227/chartnav-platform.git
cd chartnav-platform
cp infra/docker/.env.staging.example infra/docker/.env.staging
# edit .env.staging with real values — see docs/build/19-staging-deployment.md
```

Validate before starting anything:

```
docker compose \
  --env-file infra/docker/.env.staging \
  -f infra/docker/docker-compose.staging.yml \
  config >/dev/null
```

Failure here (`required variable X is missing`) is good: the compose file intentionally `${VAR:?}` -blocks on every critical var.

2. Deploy

```
make staging-up          # == bash scripts/staging_up.sh
make staging-verify      # == bash scripts/staging_verify.sh
```

Expected finish:

```
==> staging verify: PASS
```

If verify fails at `/ready` , the API can't reach the DB. Check in order:

1. `docker compose -f infra/docker/docker-compose.staging.yml ps` — db should be `healthy` .
2. `docker compose -f infra/docker/docker-compose.staging.yml logs --tail=100 db` .
3. `docker compose -f infra/docker/docker-compose.staging.yml logs --tail=100 api` .

If verify fails at the auth-denial metrics check, the app is up but the exception handler didn't record the denial — check the API container's logs for a Python traceback.

3. Exercising bearer mode

`staging_verify.sh` only drives `/me` + `/encounters` as an authed caller when `CHARTNAV_AUTH_MODE=header`. In bearer mode the script stops after unauth surfaces + observability checks, because it cannot mint tokens.

Manual path:

1. Pull a real JWT from your IdP (e.g. `auth0` CLI, `okta` CLI, a curl against the IdP's token endpoint).

2. Confirm the token's `email` claim (or whatever you set `CHARTNAV_JWT_USER_CLAIM` to) matches a row in `users`.

3. Run:

```bash

curl -sS -H "Authorization: Bearer $TOKEN" http://127.0.0.1:8000/me | jq

curl -sS -H "Authorization: Bearer $TOKEN" http://127.0.0.1:8000/encounters | jq 'length'

```

4. Watch `docker compose logs -f api` in another terminal; the access log should show `user_email` and `organization_id`.

If bearer validation fails, the `error_code` in the 401 body pinpoints the reason — `invalid_token`, `invalid_issuer`, `invalid_audience`, `token_expired`, `missing_user_claim`, `unknown_user`. Every one of those also writes a `security_audit_events` row.

4. Rollback

The contract: "image tag rollback restores behavior, not data."

```
# Pin a previously-published tag and restart the API only.
make staging-rollback TAG=v0.0.9
make staging-verify
```

What the script does (honest):

1. Rewrites `CHARTNAV_IMAGE_TAG=` in `infra/docker/.env.staging`.

2. `docker compose pull api`.

3. `docker compose up -d api`.

4. Polls `/ready` for ≤40s.

What it does **not** do:

- Reverse any migrations. If release `N` applied a destructive migration (dropped a column, dropped a table), going back to `N-1`'s image will **not** restore data. Forward-only migrations are the policy — plan accordingly before tagging a release.
- Blue/green. There's one API replica. During the restart, `/ready` will go red for a handful of seconds; the reverse proxy should drain accordingly.

If rollback fails `/ready`, you're worse off than before rollback and need to triage:

1. `docker compose logs --tail=200 api`.

2. Pin the **current broken** tag back (`make staging-rollback TAG=<the tag you started from>`) — sometimes the issue is "the image didn't actually pull".

3. If the DB is broken (not the image), the recovery path is a DB restore from your backup. That is out-of-band of this repo; document it in your infra runbook.

5. Health / observability quick checks

```
# liveness + readiness
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/ready

# metrics snapshot
curl -fsS http://127.0.0.1:8000/metrics | grep -E '^(chartnav_requests_total|chartnav_auth_denied_total|chartnav_rate_limited_

# recent audit rows (inside the db container)
docker compose -f infra/docker/docker-compose.staging.yml \
  --env-file infra/docker/.env.staging \
```

```
  exec -T db psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -c "SELECT id, event_type, actor_email, path, error_code, created_at \
      FROM security_audit_events ORDER BY id DESC LIMIT 20;"

# follow logs (JSON; pipe through jq for readability)
docker compose -f infra/docker/docker-compose.staging.yml \
  --env-file infra/docker/.env.staging \
  logs -f api | sed 's/^[^{]*//' | jq -cR 'fromjson? | select(.) | .'
```

See `20-observability.md` for the full field reference.

6. Teardown

```
# soft — keep the DB volume
make staging-down

# hard — DROP ALL DATA
cd infra/docker && docker compose \
  --env-file .env.staging -f docker-compose.staging.yml down -v
```

Never run the hard teardown on a shared staging host without confirming with whoever else uses it.

7. Common failures, honest fixes

Symptom	Likely cause	Fix
`required variable CHARTNAV_JWT_JWKS_URL is missing`	`CHARTNAV_AUTH_MODE=bearer` but JWT config blank in `.env.staging`	Fill in the three JWT vars or set `AUTH_MODE=header` for triage.
`db` never becomes healthy	Volume permissions / old volume with wrong password	`docker volume rm docker_chartnav_staging_pgdata` **iff** you accept data loss.
Smoke passes but app UX is broken	Stale frontend bundle vs. backend contract	Re-pin frontend asset (out of scope of this compose) — the API is fine.
429 everywhere	`CHARTNAV_RATE_LIMIT_PER_MINUTE` too low for a load test	Temporarily raise; remember it's per-process, per-path, per-IP.
`invalid_issuer` / `invalid_audience` flood	Wrong `CHARTNAV_JWT_ISSUER` or `_AUDIENCE` vs. the IdP's tokens	Inspect a sample token's `iss` / `aud`; align env.

8. What this runbook intentionally does not cover

- Backup & restore. That's your infra concern; the app is stateless beyond Postgres.
- DNS / TLS / edge routing.
- Disaster recovery beyond image rollback.
- Secret rotation. Rotate in your secret store, then `make staging-up` re-reads env on container restart.

When any of these become first-class ChartNav concerns, they graduate to their own build-log phase and doc section.

Diagram — System architecture

System Architecture

```
flowchart LR subgraph Web["Frontend (apps/web, Vite + React + TS)"] UI["App.tsx
list · detail · timeline · actions"] ApiClient["api.ts
typed client + ApiError"] Ident["identity.ts
localStorage dev caller"] end subgraph Runtime["Backend (apps/api, FastAPI)"] FastAPI["main.py
+ exception handler → audit"] RID["RequestIdMiddleware"] Access["AccessLogMiddleware
(JSON logs)"] Rate["RateLimitMiddleware
(60s window)"] Audit[("security_audit_events")] Metrics["metrics.py
(chartnav_* counters)"] Ready["/ready (DB ping)"] Config["config.py
env-driven settings"] Authn["require_caller
header · bearer (PyJWKClient)"] Authz["authz.py RBAC"] Router["routes.py"] SM["State Machine"] DB["db.py · SA Core"] Engine[(SA Engine)]
end subgraph Storage SQLite[("SQLite
apps/api/chartnav.db")] PG[("Postgres 16")] end subgraph Deploy Dockerfile["Dockerfile
non-root · healthcheck"] Entry["entrypoint.sh
migrate + exec"] Compose["docker-compose.prod.yml"] StageCompose["docker-compose.staging.yml
pinned tag · /ready healthcheck"] StageScripts["staging_up · staging_verify
staging_rollback"] end subgraph Tooling Alembic["Alembic
DATABASE_URL-aware"] Seed["scripts_seed.py"] PyTest["pytest (28)"] VitestSuite["vitest (12)"] Playwright["Playwright E2E (8)"]
Smoke["scripts/smoke.sh"] Verify["scripts/verify.sh"] PgVerify["scripts/pg_verify.sh"] DocBuild["scripts/build_docs.py"]
ReleaseScript["scripts/release_build.sh"] Make["Makefile
install · verify · pg-verify · dev · web-* · e2e · release-build"] CI[".github/workflows/
ci.yml · release.yml"] end UI --> ApiClient ApiClient -->|"HTTP + X-User-Email or Bearer"| RID RID --> Access Access --> Rate Rate --> FastAPI
FastAPI -. audits denials .-> Audit FastAPI -. bumps .-> Metrics Access -. observes .-> Metrics Rate -. observes .-> Metrics FastAPI --> Ready
Metrics -->|"/metrics text"| ApiClient Ident -. stored caller .-> ApiClient FastAPI --> Authn Authn --> Authz Authz --> Router Router --> SM
Router --> DB DB --> Engine Engine --> SQLite Engine --> PG Config -.-> Authn Config -.-> DB Alembic -.-> Engine Seed -.-> DB PyTest -.->
FastAPI VitestSuite -.-> UI Playwright -.->|"live browser"| UI Playwright -.->|"live HTTP"| FastAPI Smoke -.-> FastAPI Verify --> Make PgVerify -
-> Make DocBuild --> Make Make --> Alembic Make --> Seed Make --> PyTest Make -. web-dev · web-build .-> UI CI --> Make Dockerfile -->
Entry Entry --> Alembic Entry --> FastAPI Compose --> Dockerfile Compose --> PG
```

Diagram — Encounter status machine

Encounter Status State Machine

stateDiagram-v2 [*] --> scheduled [*] --> in_progress : walk-in scheduled --> in_progress in_progress --> draft_ready draft_ready --> review_needed draft_ready --> in_progress : rework review_needed --> completed review_needed --> draft_ready : kick back completed --> [*]

Legend: any edge not shown is rejected at `POST /encounters/{id}/status`

with HTTP 400 `invalid_transition`.

Diagram — ER

ER Diagram

erDiagram organizations ||--o{ locations : "has" organizations ||--o{ users : "employs (identity + role)" organizations ||--o{ encounters : "owns" locations ||--o{ encounters : "hosts" encounters ||--o{ workflow_events : "emits" security_audit_events { int id PK string event_type string request_id string actor_email int actor_user_id int organization_id string path string method string error_code text detail string remote_addr datetime created_at } organizations { int id PK string name string slug UK datetime created_at } locations { int id PK int organization_id FK string name datetime created_at } users { int id PK int organization_id FK string email UK string full_name string role "admin | clinician | reviewer" datetime created_at } encounters { int id PK int organization_id FK int location_id FK string patient_identifier string patient_name string provider_name string status datetime scheduled_at datetime started_at datetime completed_at datetime created_at } workflow_events { int id PK int encounter_id FK string event_type text event_data datetime created_at }

Seeded tenants & users

org_id	slug	email	role
1	`demo-eye-clinic`	admin@chartnav.local	admin
1	`demo-eye-clinic`	clin@chartnav.local	clinician
1	`demo-eye-clinic`	rev@chartnav.local	reviewer
2	`northside-retina`	admin@northside.local	admin
2	`northside-retina`	clin@northside.local	clinician

`users.email` is the authentication key consumed from `X-User-Email`.

`users.role` is the RBAC key consumed by `app.authz`.

`users.organization_id` is the authoritative source of scope; never

derived from client input.

Diagram — API / data flow

API / Data Flow & CI

UI → Backend request path

sequenceDiagram autonumber participant U as Operator (browser) participant W as apps/web (App.tsx + api.ts) participant Cfg as app.config participant Authn as require_caller participant Authz as authz gate participant R as Route handler participant SM as State Machine participant DB as SA Core → SQLite / Postgres U->>W: pick identity · change filter · click transition W->>Authn: fetch with X-User-Email: admin@chartnav.local Cfg-->>Authn: auth_mode=header Authn->>DB: SELECT user WHERE email=:email DB-->>Authn: {role, org} Authn-->>Authz: Caller Authz->>R: Caller R->>DB: scoped SQL (:name binds) DB-->>R: rows R-->>W: 200 + JSON (or 4xx {error_code, reason}) W-->>U: render list · detail · timeline · banner Note over W,R: On 4xx, api.ts wraps into ApiError(status, error_code, reason). App.tsx surfaces it verbatim as a banner.

Role-aware transition flow

sequenceDiagram autonumber participant W as App.tsx participant A as api.allowedNextStatuses() participant B as POST /encounters/{id}/status participant Authz as authz.assert_can_transition W->>A: compute buttons for (role, current) A-->>W: [next1, next2, ...] (UI hint only) W->>B: POST status=next B->>Authz: validate state-machine edge + role edge alt allowed Authz-->>B: ok B-->>W: 200 updated encounter W->>W: refresh detail + events + list; green banner else role forbidden Authz-->>W: 403 role_cannot_transition W->>W: red banner; UI affordances unchanged until next refresh end

CI gate flow

flowchart TD PR[["push / PR"]] --> SQL[backend-sqlite] SQL --> PG[backend-postgres] SQL --> DK[docker-build] SQL --> DOC[docs] subgraph "backend-sqlite" A1[pip install -e .dev,postgres] --> A2[alembic upgrade] A2 --> A3[seed x2] A3 --> A4[pytest 28] A4 --> A5[scripts/verify.sh] end subgraph "backend-postgres (service: postgres:16-alpine)" B1[install] --> B2[alembic upgrade] B2 --> B3[seed x2] B3 --> B4[boot + smoke] B4 --> B5[live status transition] end subgraph "docker-build" D1[buildx build] --> D2[run container] D2 --> D3[smoke] end subgraph "docs" X1[apt chromium] --> X2[build_docs.py] X2 --> X3[upload artifact] end

Bearer mode + audit trail (phase 10)

sequenceDiagram autonumber participant C as Client participant RID as RequestIdMiddleware participant Rate as RateLimitMiddleware participant Authn as require_caller (bearer) participant JWKS as JWKS URL (PyJWKClient) participant R as Route participant EH as HTTPException handler participant Audit as security_audit_events C->>RID: GET /me
Authorization: Bearer RID->>Rate: request + request.state.request_id Rate->>Authn: request (within window) Authn->>JWKS: get_signing_key_from_jwt(token) JWKS-->>Authn: public key Authn->>Authn: jwt.decode — sig + iss + aud + exp alt invalid token / iss / aud / expired / missing claim / unknown user Authn-->>EH: HTTPException(status=401, error_code=...) EH->>Audit: record(event_type, request_id, path, method, remote_addr, ...) EH-->>C: 401 + X-Request-ID else valid Authn->>Authn: SELECT users WHERE email = claim Authn-->>R: Caller(user_id, role, organization_id) R-->>C: 200 + X-Request-ID end

Staging deploy flow (phase 11)

flowchart LR Op[Operator] --> Up[scripts/staging_up.sh] Up --> V[docker compose config] V --> UP[docker compose up -d] UP --> API[api container
pinned ghcr.io/.../chartnav-api:$TAG] UP --> DB[(postgres:16-alpine)] API --> E[entrypoint.sh
alembic upgrade + uvicorn] E --> Ready["HEALTHCHECK
GET /ready"] Ready -->|ok| Proxy[reverse proxy] Op --> Verify[scripts/staging_verify.sh] Verify -->|health · ready · metrics · 401 · x-request-id ·

```
mutation · audit signal| API Op -. bad tag .-> Rollback[scripts/staging_rollback.sh PREV_TAG] Rollback --> RP[rewrite CHARTNAV_IMAGE_TAG] RP --> UPapi[docker compose up -d api] UPapi --> Ready
```

Release pipeline

```
flowchart LR TAG[["git push tag v*.*.*"]] --> RB[release.yml] RB --> V[resolve version] V --> DK[docker buildx push ghcr.io/.../chartnav-api:version + :latest] V --> RBS[scripts/release_build.sh] RBS --> API[chartnav-api-<v>.tar] RBS --> WEB[chartnav-web-<v>.tar.gz] RBS --> M[MANIFEST.txt] API --> GR[GitHub Release] WEB --> GR M --> GR DK --> OPS[operator: docker pull + compose up] GR --> OPS
```